



chapter 6

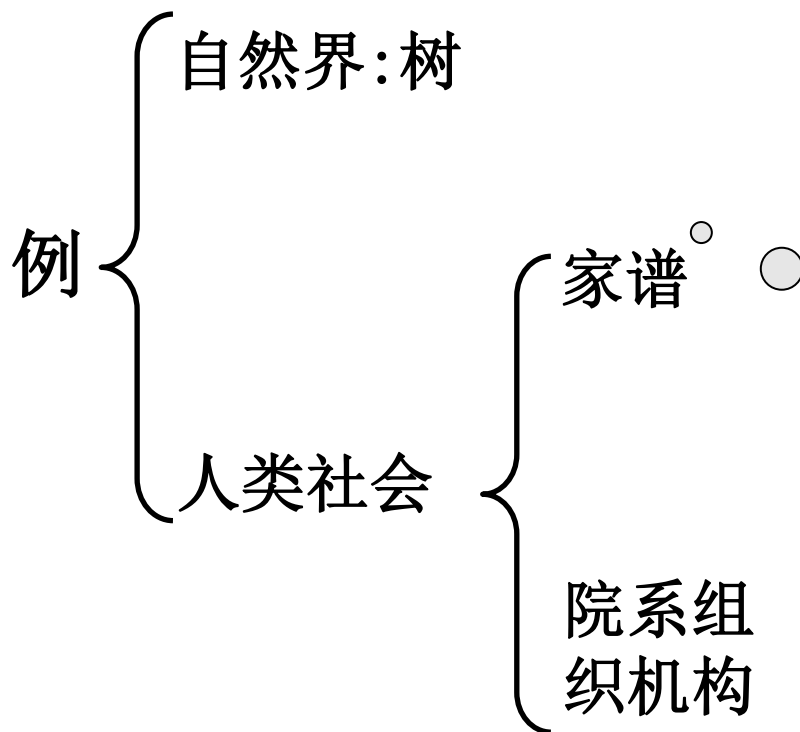
■ ■ ■ ■ ■ ■ ■ ■ ■

树与二叉树

主讲人：周瑜

- 6.1 树的基本概念
- 6.2 二叉树的定义、性质及运算
- 6.3 二叉树的存储结构
- 6.4 遍历二叉树
- 6.5 线索二叉树
- 6.6 树和森林
- 6.7 哈夫曼树、哈夫曼编码

树型结构（非线性结构） { 结点之间有分支
具有层次关系



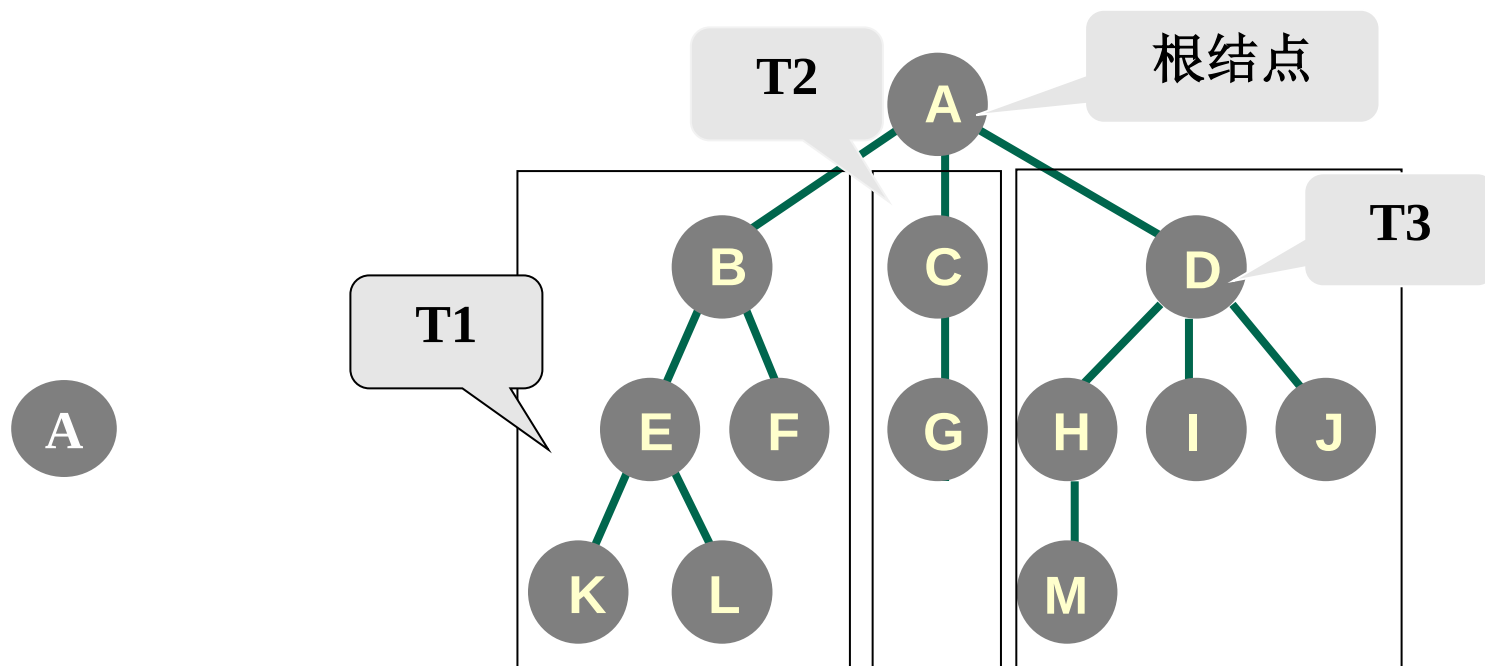
生活中的哪些实
例是树型结构呢？

树的定义

树是 $n(n \geq 0)$ 个结点的有限集。若 $n=0$ ，称为空树；若 $n>0$ ，则它满足如下两个条件：

- 有且仅有一个称之为根(Root)的结点。
- 当 $n>1$ ，除根以外的其它结点分为 $m(m>0)$ 个互不相交的有限集 $T_1, T_2, \dots, T_m$ ，其中每个集合又是一棵符合本定义棵树，并且称为根的子树。

例如



只有根结点的树

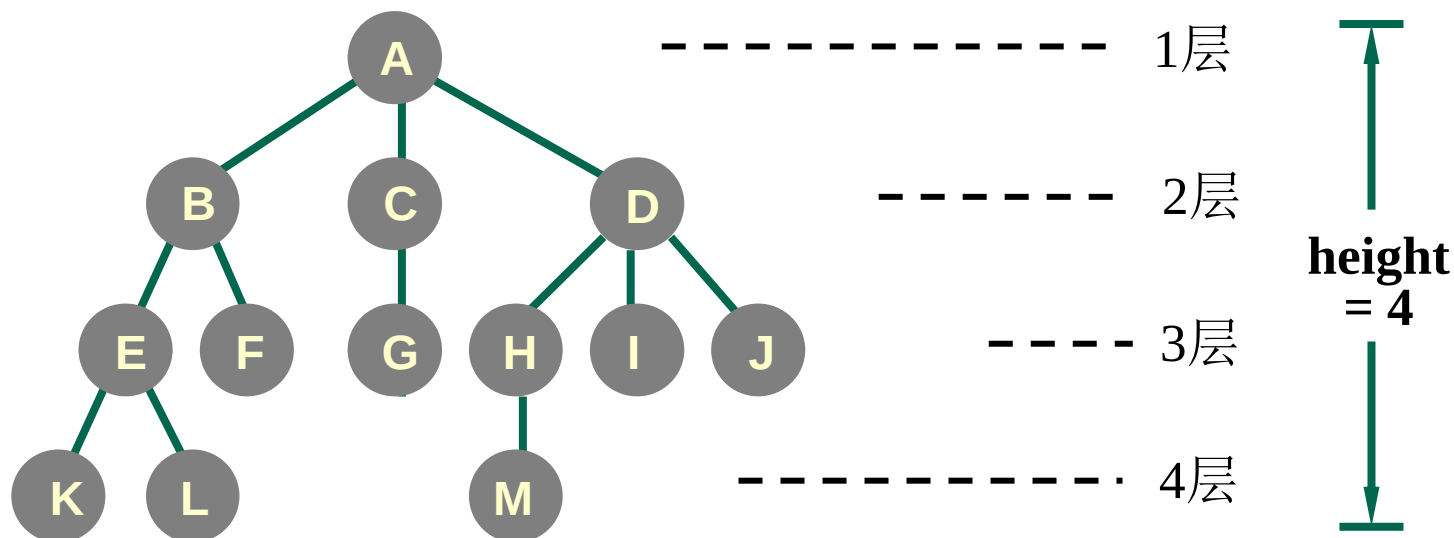
有13个结点的树

A是根；其余数据元素分成三个互不相交的子集，
 $T1=\{B,E,F,K,L\}$ ； $T2=\{C,G\}$ ； $T3=\{D,H,I,J,M\}$ ，
 $T1,T2,T3$ 都是根A的子树，且本身也是一棵树。

树结构和线性结构作如下对照：

线性结构	树结构
存在唯一的没有前驱的“首元素”	存在唯一的没有前驱的“根结点”
存在唯一的没有后继的“尾元素”	存在多个没有后继的“叶子”
其余元素均存在唯一的“前驱元素”和唯一的“后继元素”	其余结点均存在唯一的“前驱（双亲）结点”和多个“后继（孩子）结点”

树的基本术语



* 结点

* 结点的度

* 叶子结点

* 分支结点

* 儿子结点

* 父亲结点

* 兄弟结点

* 祖先结点

* 树的度

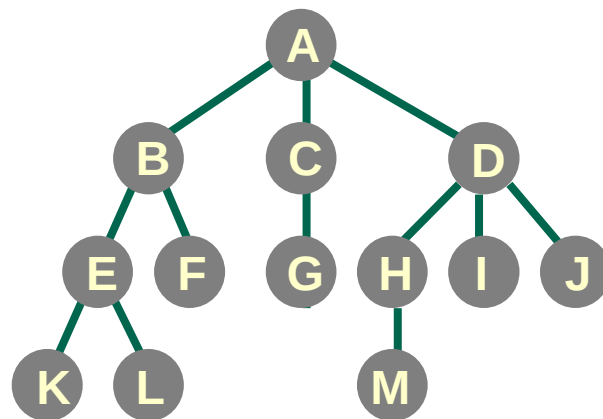
* 结点的层次

* 树的深度

* 森林

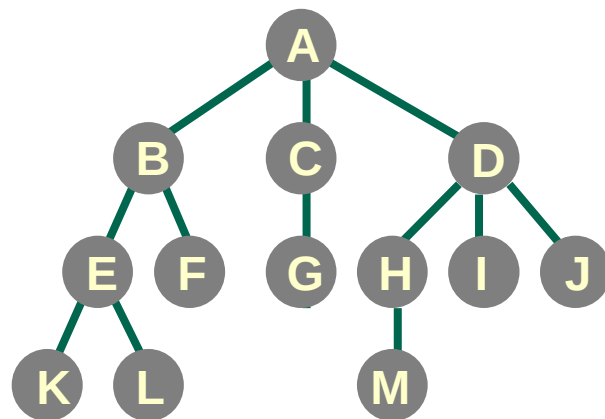
树的基本术语

- 根 —— 即根结点(没有前驱)
- 叶子 —— 即终端结点(没有后继)
- 森林 —— 指m棵不相交的树的集合(例如删除A后的子树个数)



- 有序树 —— 结点各子树从左至右有序，不能互换（左为第一）
- 无序树 —— 结点各子树可互换位置。

树的基本术语



双亲 —— 即上层的那个结点
(直接前驱)

孩子 —— 即下层结点的子树的根(直接后继)

兄弟 —— 同一双亲下的同层结点 (孩子之间互称兄弟)

堂兄弟 —— 即双亲位于同一层的结点 (但并非同一双亲)

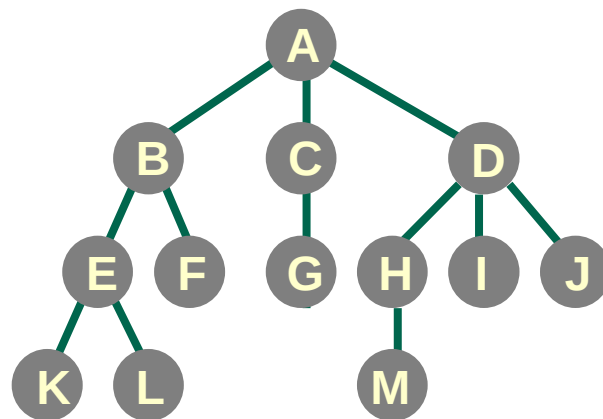
祖先 —— 即从根到该结点所经分支的所有结点

子孙 —— 即该结点下层子树中的任一结点

树的基本术语

结点 —即树的数据元素
结点的度 —结点挂接的子树数

(有几个直接后继就是几度，亦称“次数”)



结点的层次 —从根到该结点的层数（根结点算第一层）

终端结点 —即度为0的结点，即叶子

分支结点 —即度不为0的结点（也称为内部结点）

树的度 —所有结点度中的最大值 ($\text{Max}\{\text{各结点的度}\}$)

树的深度 —指所有结点中最大的层数 ($\text{Max}\{\text{各结点的层次}\}$)
(或高度)

6.1

树的基本概念

结点A的度: 3
结点B的度: 2
结点M的度: 0

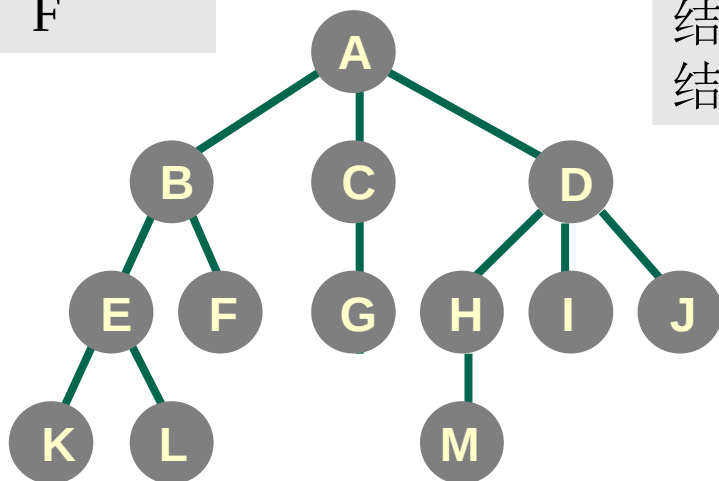
叶子: K, L, F, G, M, I, J

结点I的双亲: D
结点L的双亲: E

结点A的孩子: B, C, D
结点B的孩子: E, F

树的度: 3

结点B, C, D为兄弟
结点K, L为兄弟



树的深度: 4

结点A的层次: 1
结点M的层次: 4

结点F, G为堂兄弟
结点A是结点F, G的祖先

树的性质：

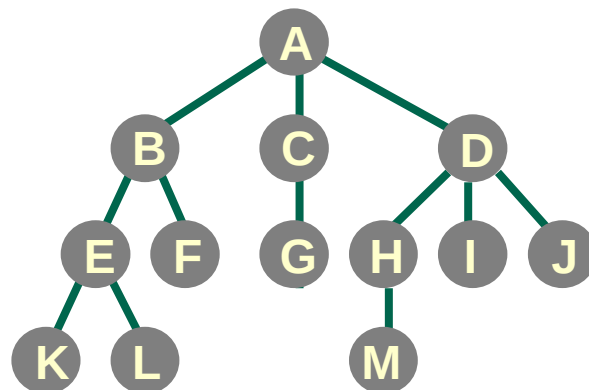
性质1：树中的节点数 n 等于所有节点的度数加1
(在一颗树中，度之和=分支数，而分支数= $n-1$)

性质2：度为 m 的树中第 i 层上至多有 m^{i-1} 个节点($i \geq 1$)

性质3：高度为 h 的度为 m 的树至多有 $\frac{m^h-1}{m-1}$ 个节点。

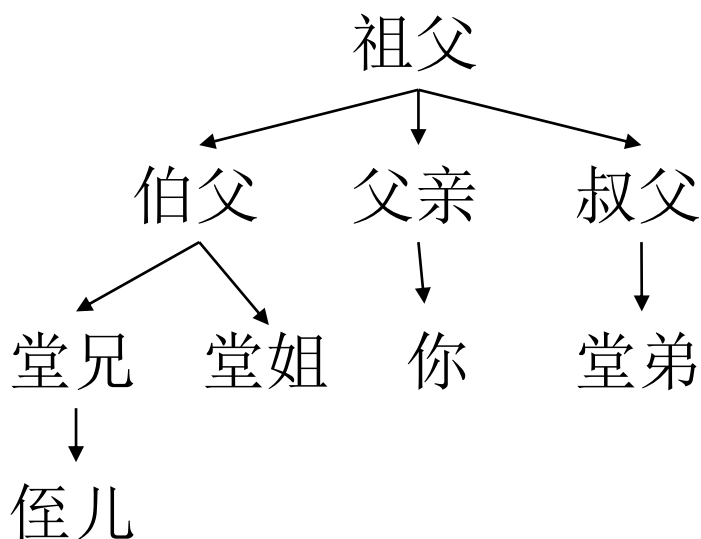
性质4：具有 n 个节点的度为 m 的树的最小高度为 $\lceil \log_m(n(m-1)) + 1 \rceil$ 。

树的度：3

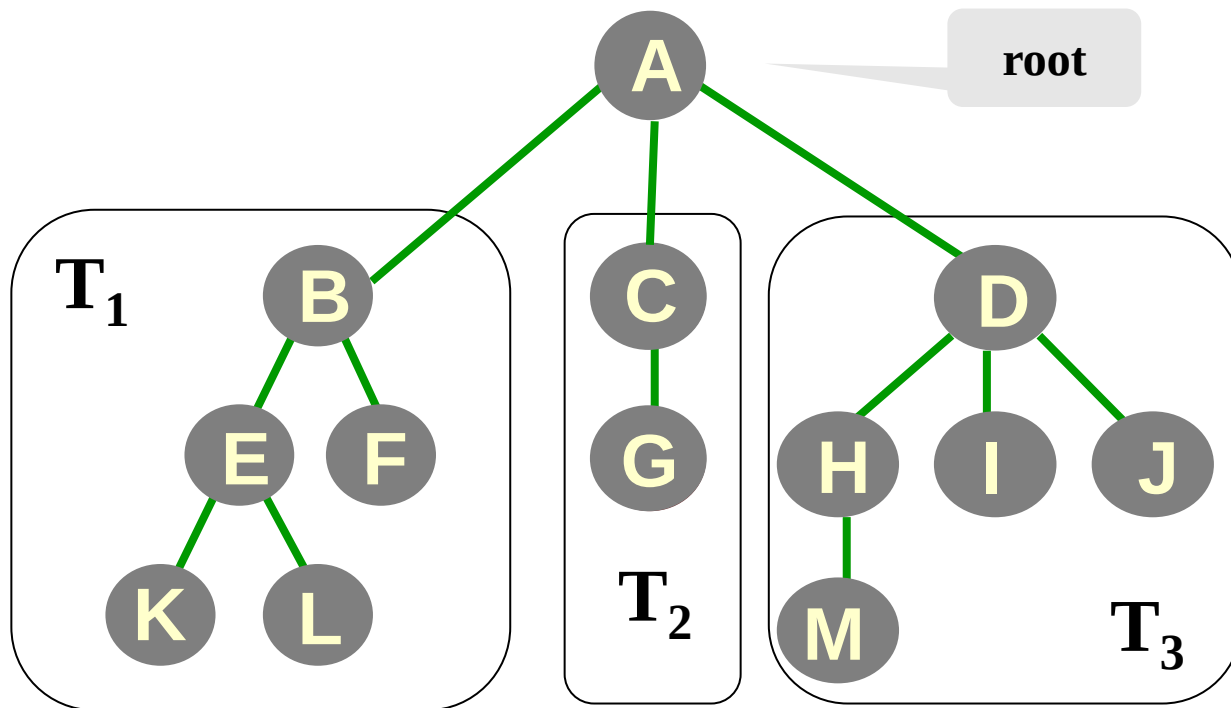


有序树 子树之间存在确定的次序关系

无序树 子树之间不存在确定的次序关系



家族树就属于有序树。

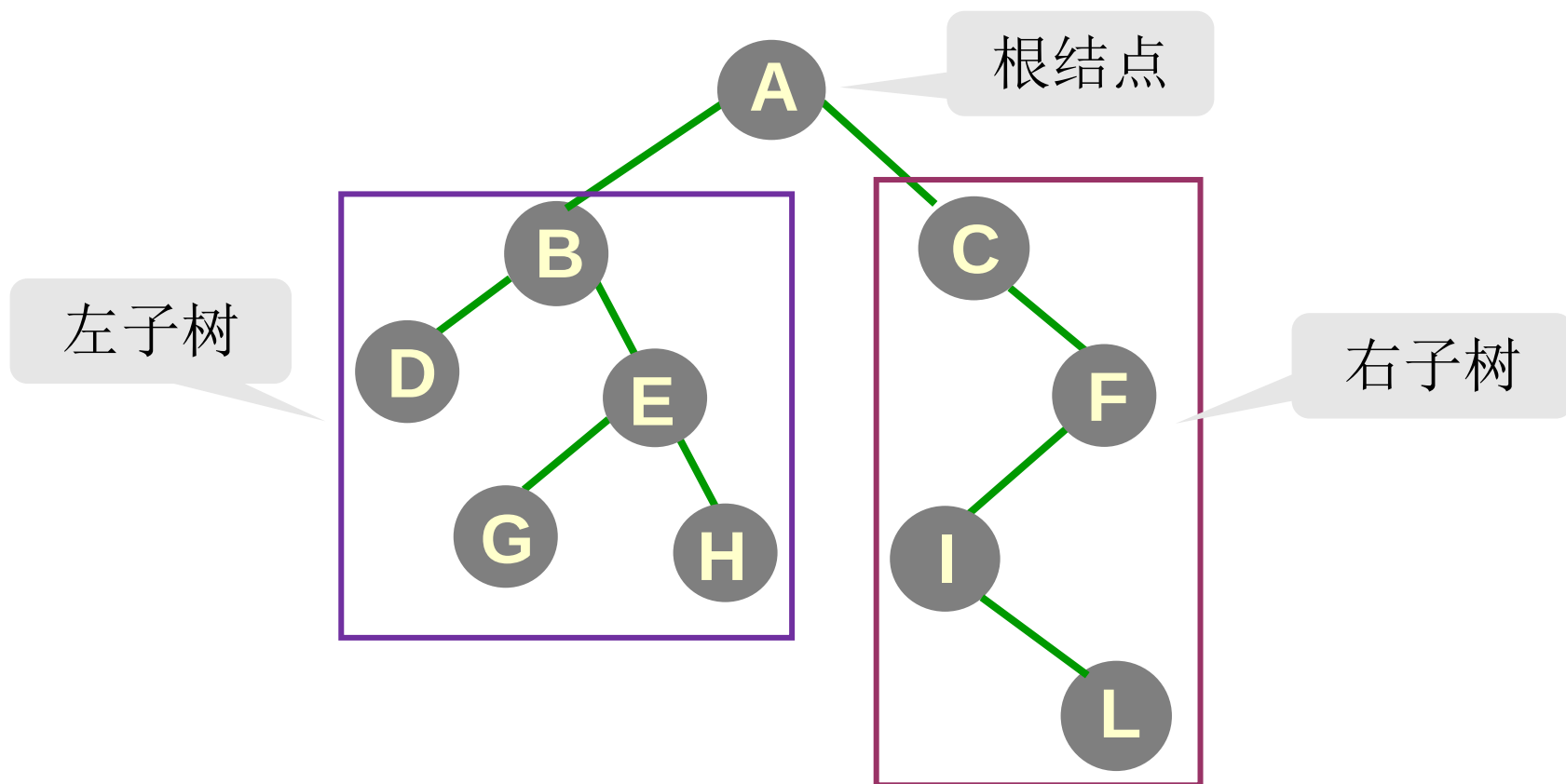


森林 是 $m(m \geq 0)$ 棵互不相交的树的集合

给森林中的各子树加上一个父亲结点, 森林就变成了树。

- 6.1 树的基本概念
- 6.2 二叉树的定义、性质及运算
- 6.3 二叉树的存储结构
- 6.4 遍历二叉树
- 6.5 线索二叉树
- 6.6 树和森林
- 6.7 哈夫曼树、哈夫曼编码

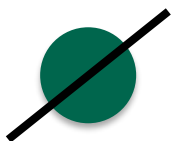
二叉树或为空树，或是由一个根结点加上两棵分别称为左子树和右子树的、互不交叉的二叉树组成。



为何要重点研究每结点最多只有两个“叉”的树？

- ✓二叉树的结构最简单，规律性最强；
- ✓可以证明，所有树都能转为唯一对应的二叉树，不失一般性。

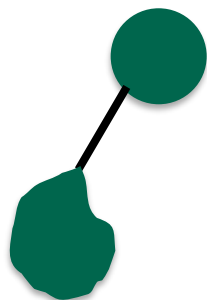
二叉树的五种基本形态



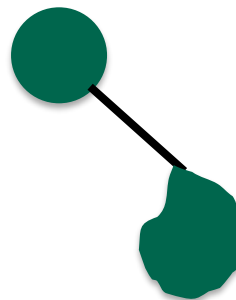
(a)
空二叉树



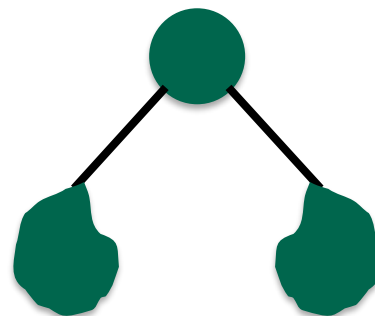
(b)
根和空的
左右子树



(c)
根和左子树



(d)
根和右子树



(e)
根和左右子树

注：虽然二叉树与树概念不同，但有关树的基本术语对二叉树都适用。

由 n 个节点构成的二叉树的形态总数为 $\frac{C_{2n}^n}{n+1}$

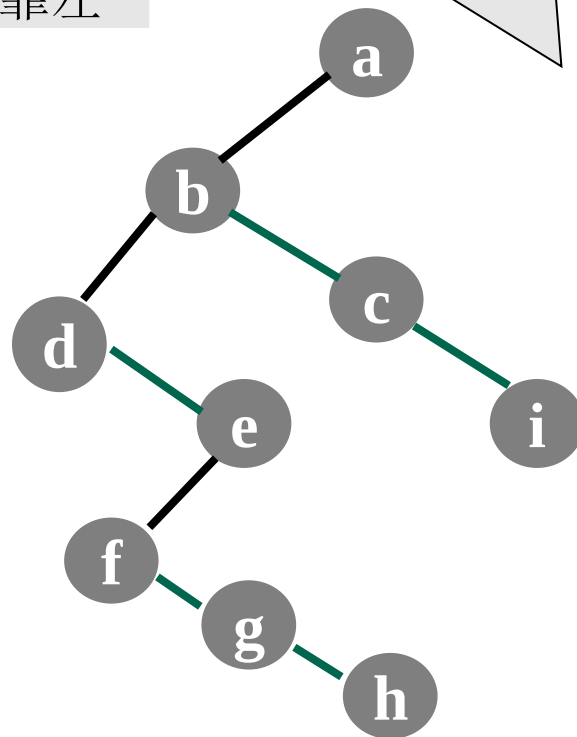
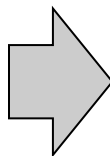
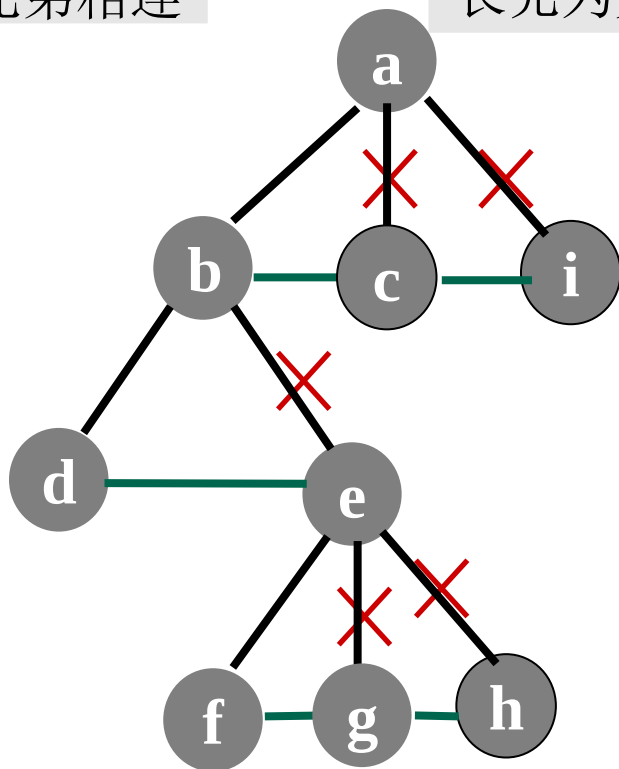
树转二叉树举例：

方法：加线—抹线—旋转

兄弟相连

长兄为父

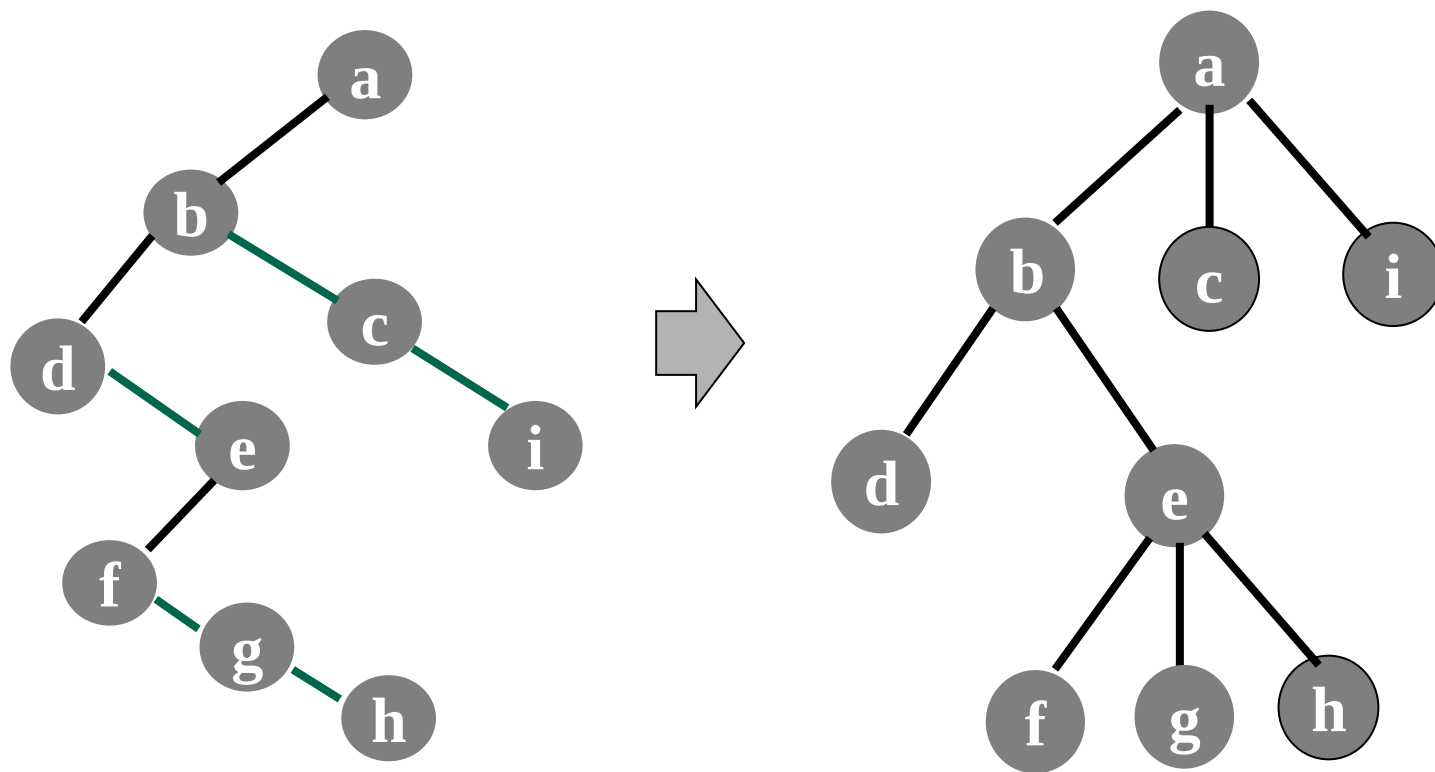
孩子靠左



特点是？
根结点没有右孩子！

二叉树怎样还原为树？

要点：逆操作，把所有右孩子变为兄弟！



二叉树性质1

在二叉树的第 i 层上至多有 2^{i-1} 个结点。

($i \geq 1$)

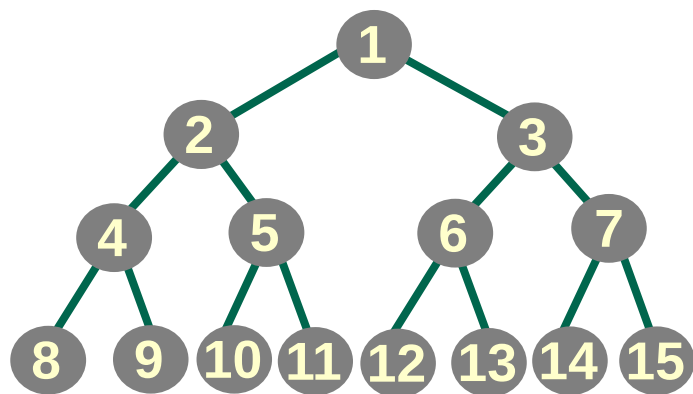
证明：

当 $i=1$ 时，只有根结点， $2^{i-1}=2^0=1$ 。

假设对所有 $j, i > j \geq 1$ ，命题成立，即第 j 层上至多有 2^{j-1} 个结点。

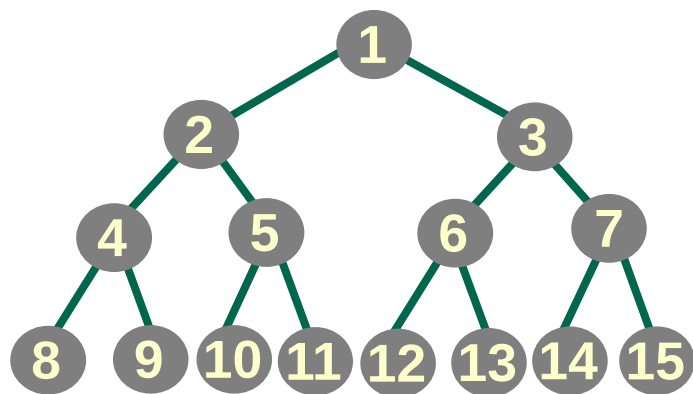
由归纳假设第 $i-1$ 层上至多有 2^{i-2} 个结点。

二叉树的每个结点的度至多为2，故在第 i 层上的最大结点数为第 $i-1$ 层上的最大结点数的2倍，即 $2 \times 2^{i-2} = 2^{i-1}$ 。



二叉树性质2

深度为 k 的二叉树至多有 2^k-1 个结点，
其中 $k \geq 1$ 。

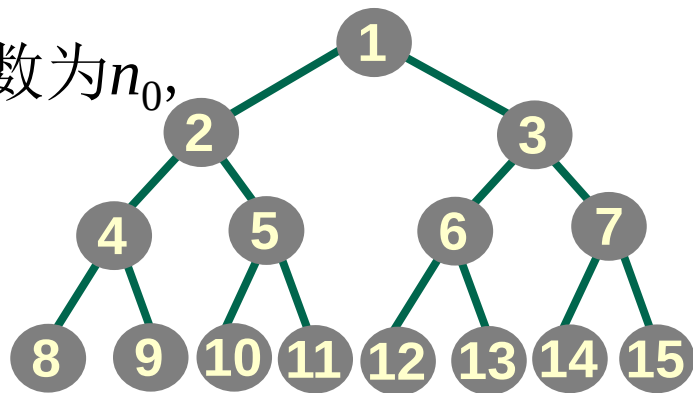


证明：由性质1可见，深度为 k 的二叉树的最大结点数为

$$\sum_{i=1}^k 2^{i-1} = 2^0 + 2^1 + \dots + 2^{k-1} = 2^k - 1$$

二叉树性质3

对任何一棵二叉树T, 如果其叶子结点数为 n_0 , 度为2的结点数为 n_2 , 则 $n_0 = n_2 + 1$ 。



证明: $n = n_0 + n_1 + n_2$ ①

设 e 为树的分支总数, 则 $e = n - 1$ 。这些分支由度为1或2的节点射出, 所以:

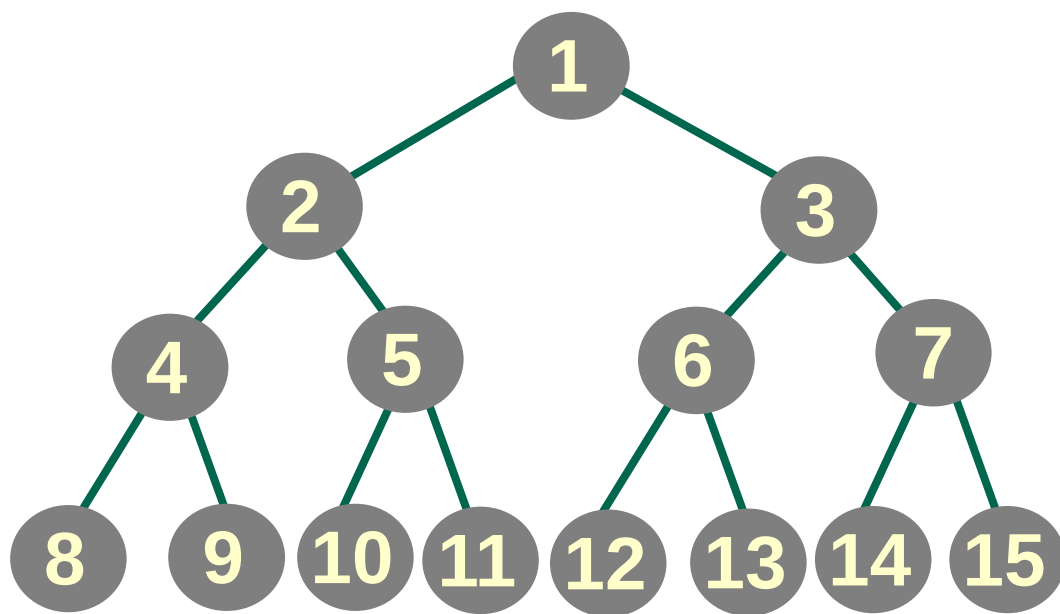
$$e = n - 1 = 2n_2 + n_1 \quad ②$$

因此, $2n_2 + n_1 = n_0 + n_1 + n_2 - 1$

$$n_0 = n_2 + 1$$

两种特殊形态的二叉树——满二叉树 (Full Binary Tree)

定义1: 一棵深度为 k 且有 2^k-1 个结点的二叉树称为满二叉树。

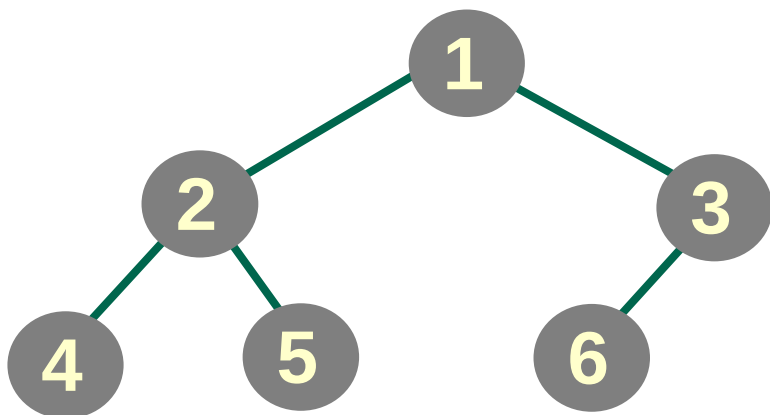


特点: 每一层上的结点数都达到最大, 叶子全部在最底层

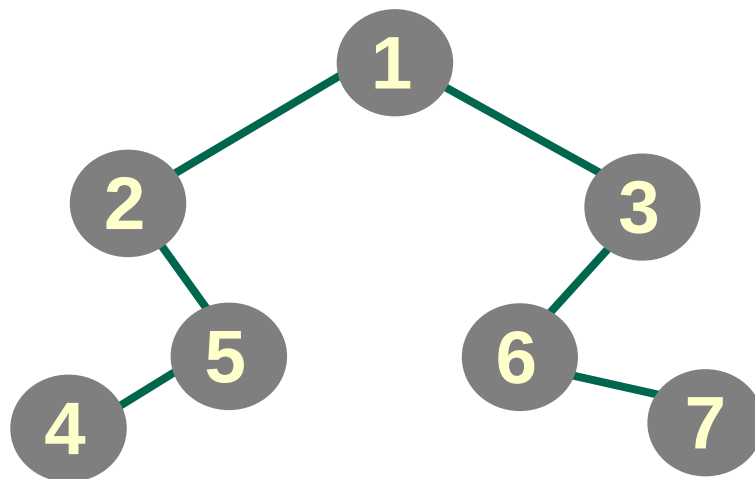
两种特殊形态的二叉树——完全二叉树 (Complete Binary Tree)

若设二叉树的深度为 h ，除第 h 层外，其它各层($0 \sim h-1$)的结点数都达到最大值，第 h 层**从右向左**连续缺若干结点。

完全二叉树

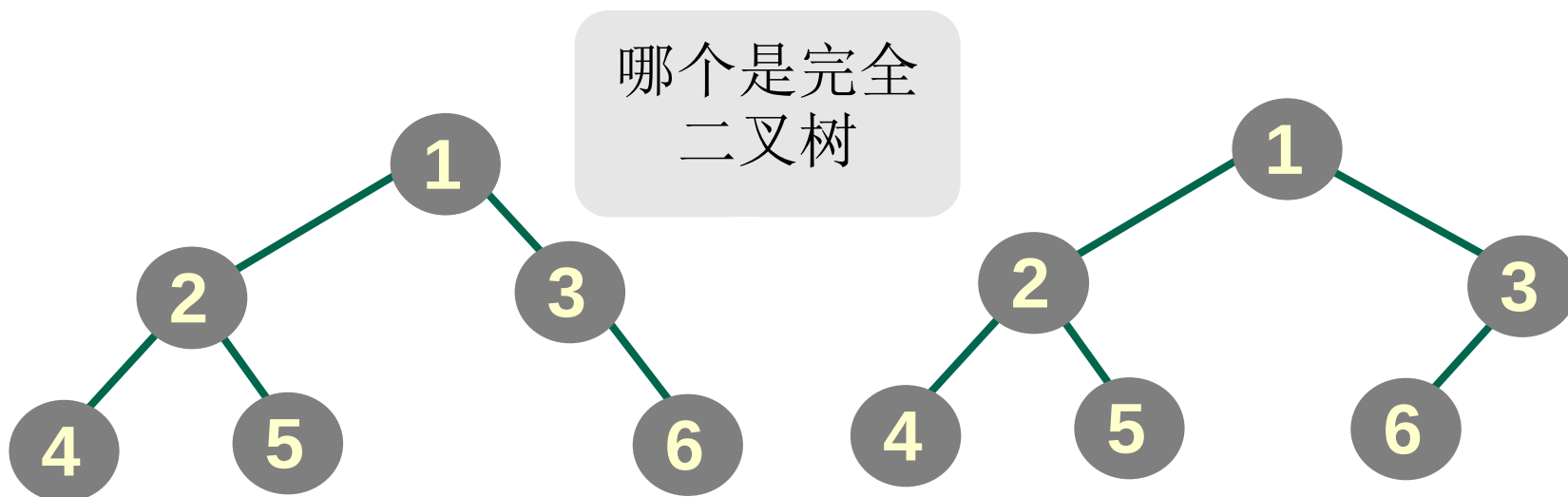


非完全二叉树



二叉树性质4

若对含 n 个结点的完全二叉树从上到下且从左至右进行1至 n 的编号，则对完全二叉树中任意一个编号为 i 的结点：



6.1

树的基本概念

6.2

二叉树的定义、性质及运算

6.3

二叉树的存储结构

6.4

遍历二叉树

6.5

线索二叉树

6.6

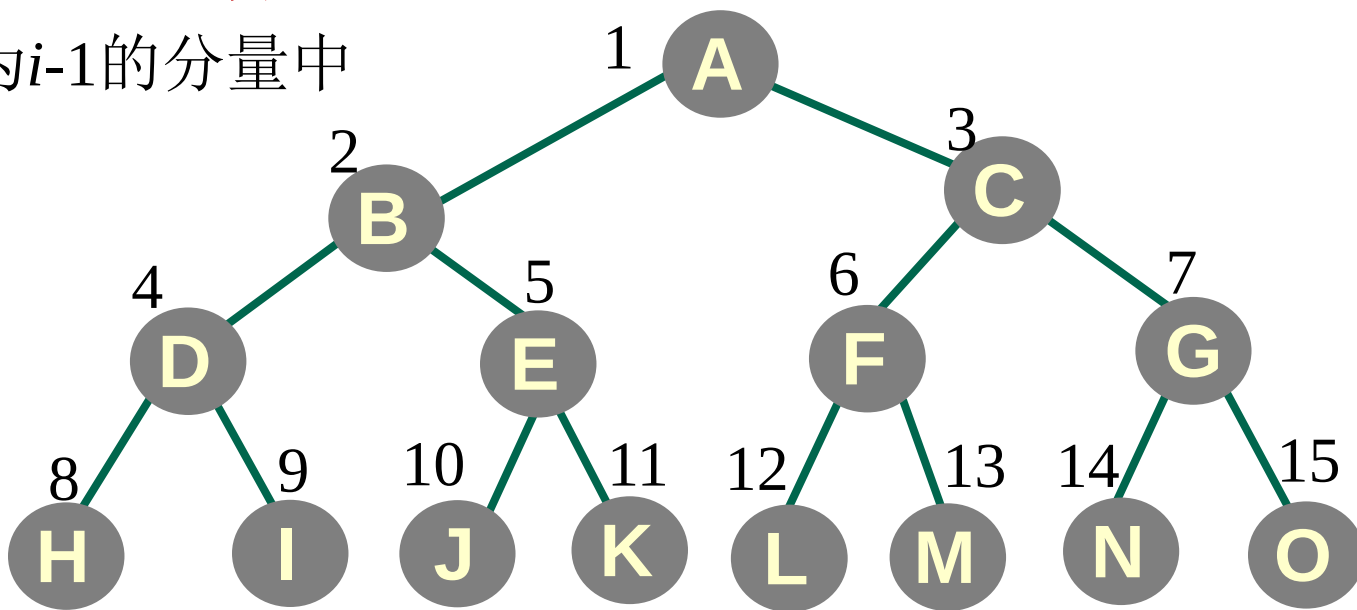
树和森林

6.7

哈夫曼树、哈夫曼编码

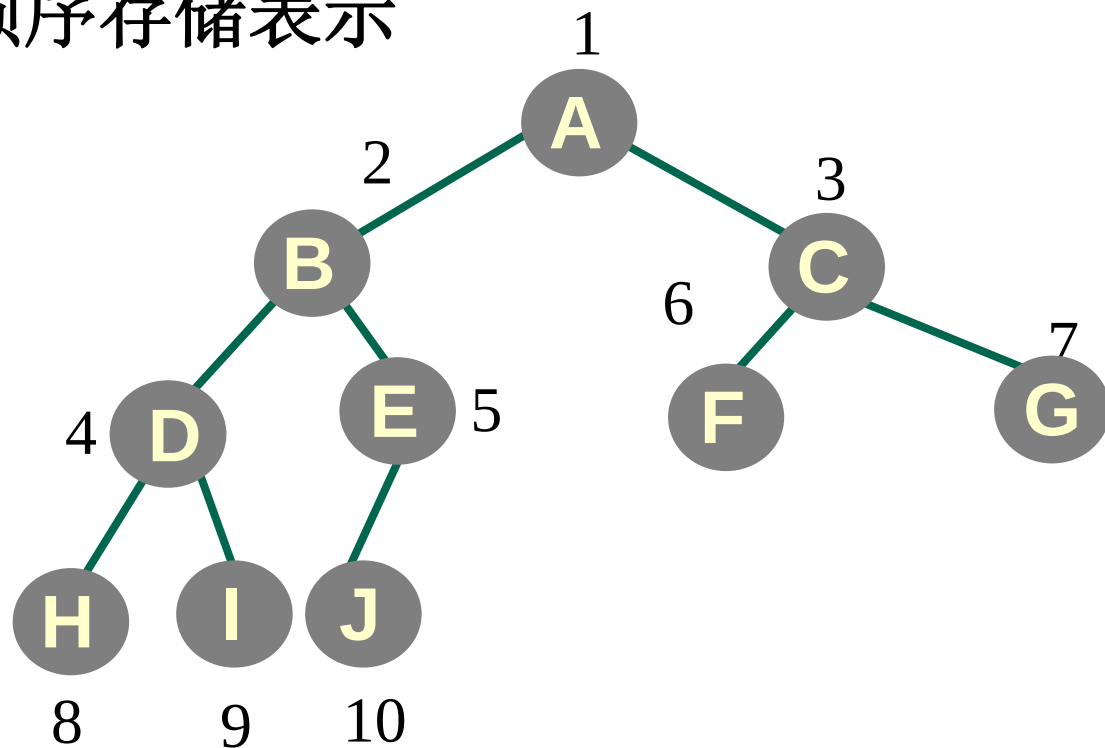
一、顺序存储表示

用一组地址连续的存储单元存储二叉树中的数据元素。
将完全二叉树上编号为 i 的结点元素存储在一维数组中下标为 $i-1$ 的分量中



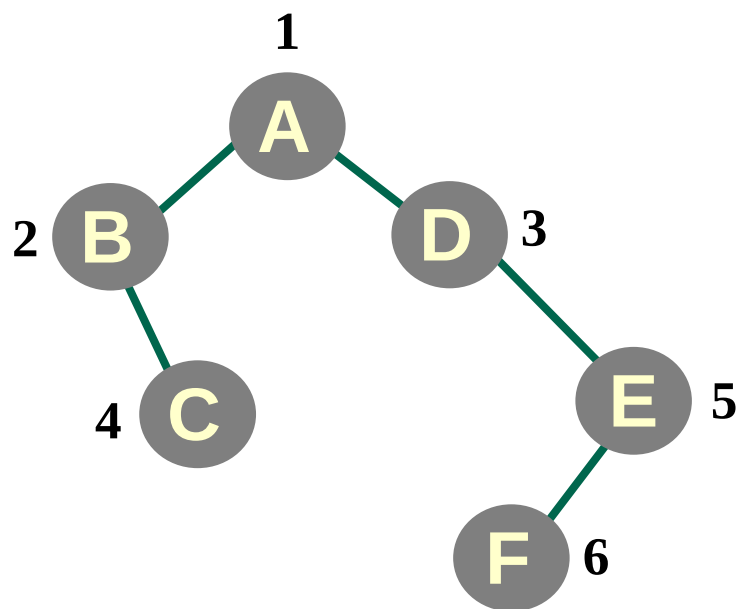
A	B	C	D	E	F	G	H	I	J	K	L	M	N	0
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

一、顺序存储表示



A	B	C	D	E	F	G	H	I	J
0	1	2	3	4	5	6	7	8	9

一、顺序存储表示

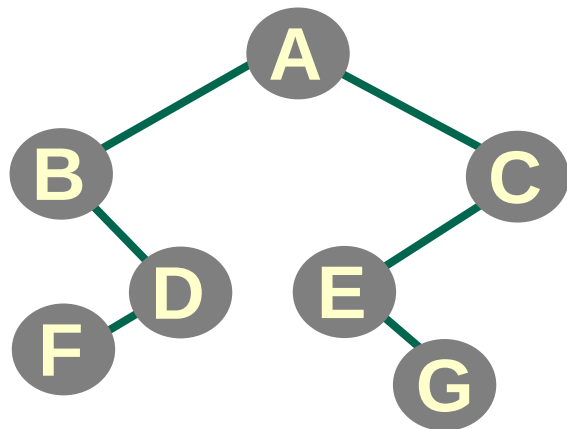


对于一般的非完全二叉树不能直接使用二叉树的顺序存储结构。

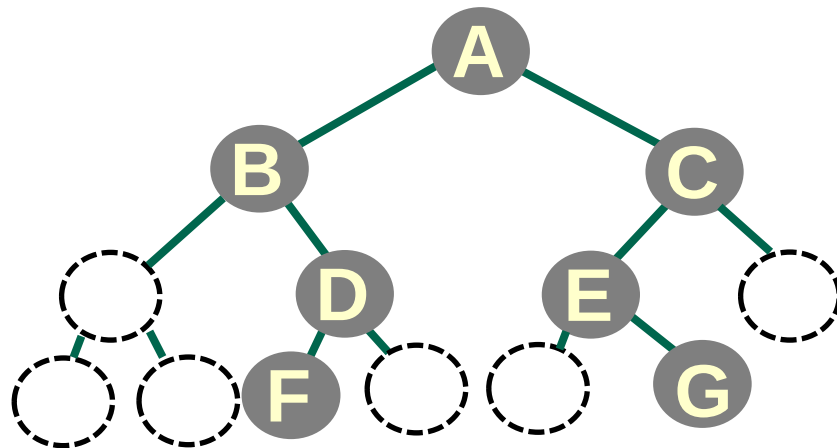
0	1	2	3	4	5
A	B	D	C	E	F

一、顺序存储表示

可以首先在非完全二叉树中增添一些并不存在的空结点使之变成完全二叉树的形态，然后再用顺序存储结构存储。



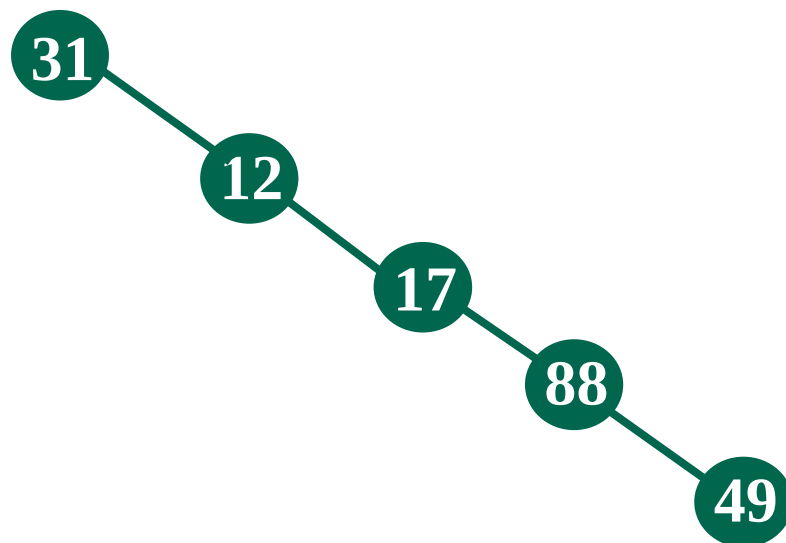
(a)一般二叉树



(b)完全二叉树形态

数组	A	B	C	^	D	E	^	^	^	F	^	^	G
下标	0	1	2	3	4	5	6	7	8	9	10	11	12

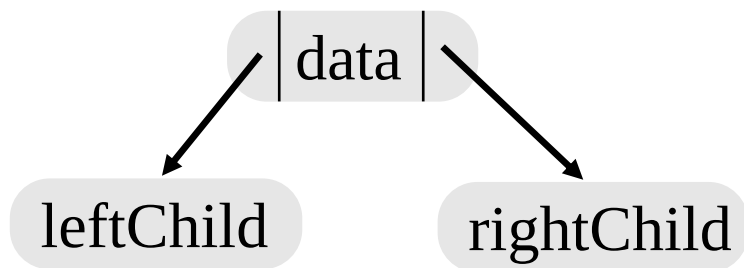
一、顺序存储表示——单支树



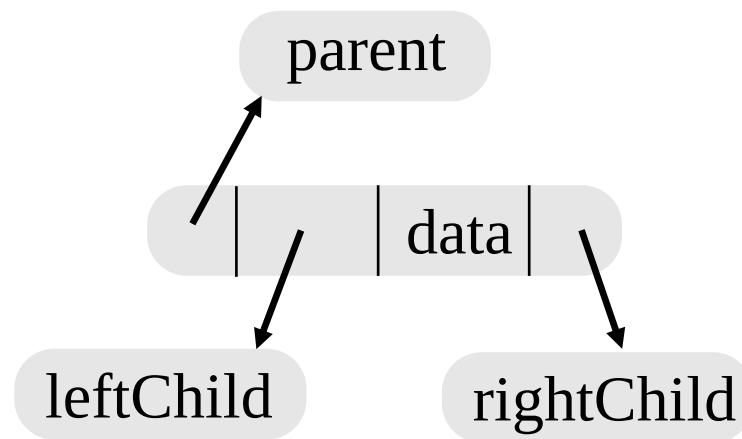
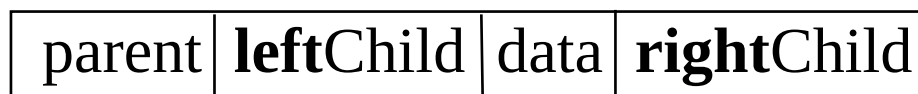
由于一般二叉树必须仿照完全二叉树那样存储，可能会浪费很多存储空间，单支树就是一个极端情况。

顺序存储结构，一般只用于完全二叉树

二、链式存储表示

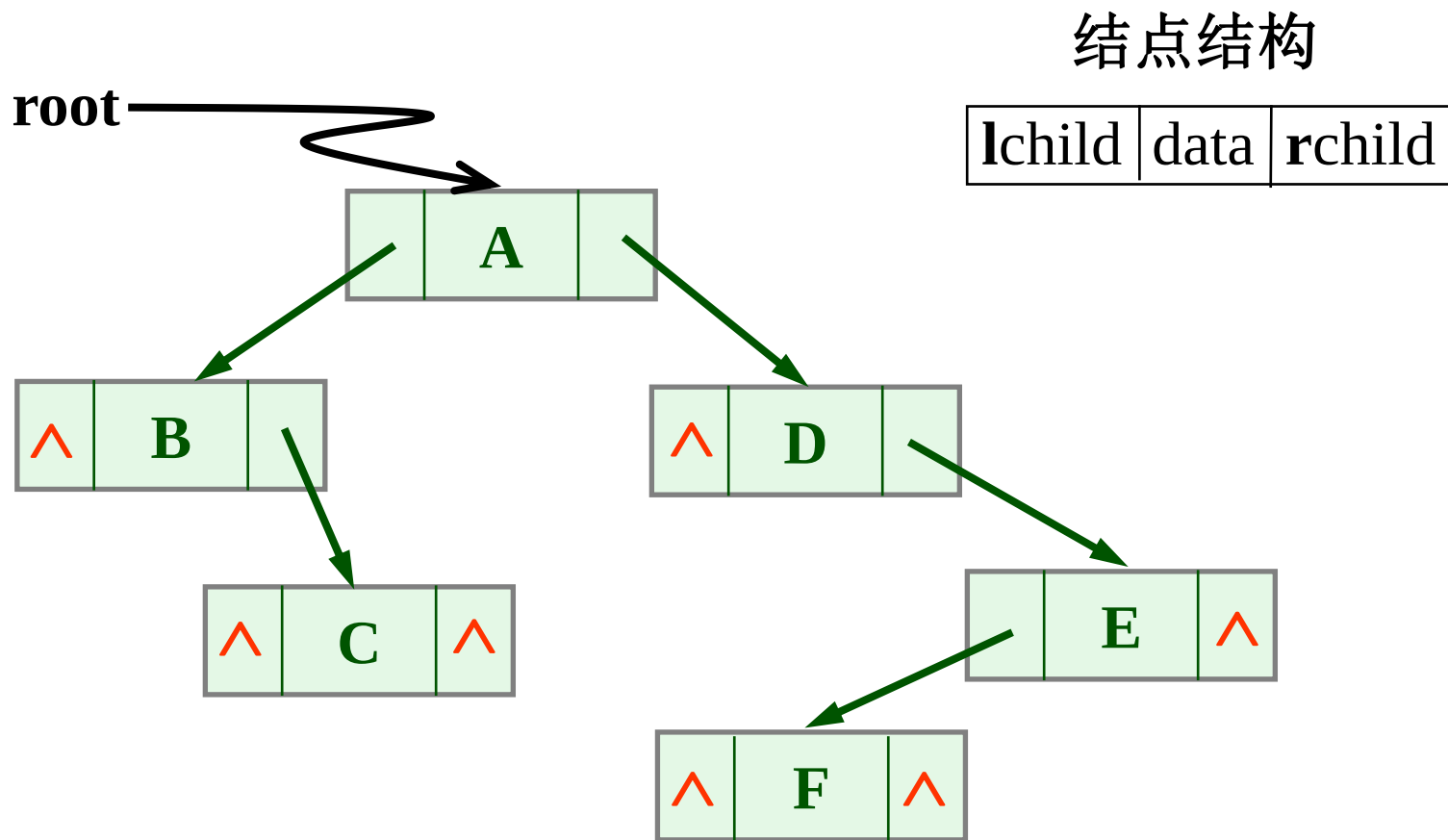


二叉链表

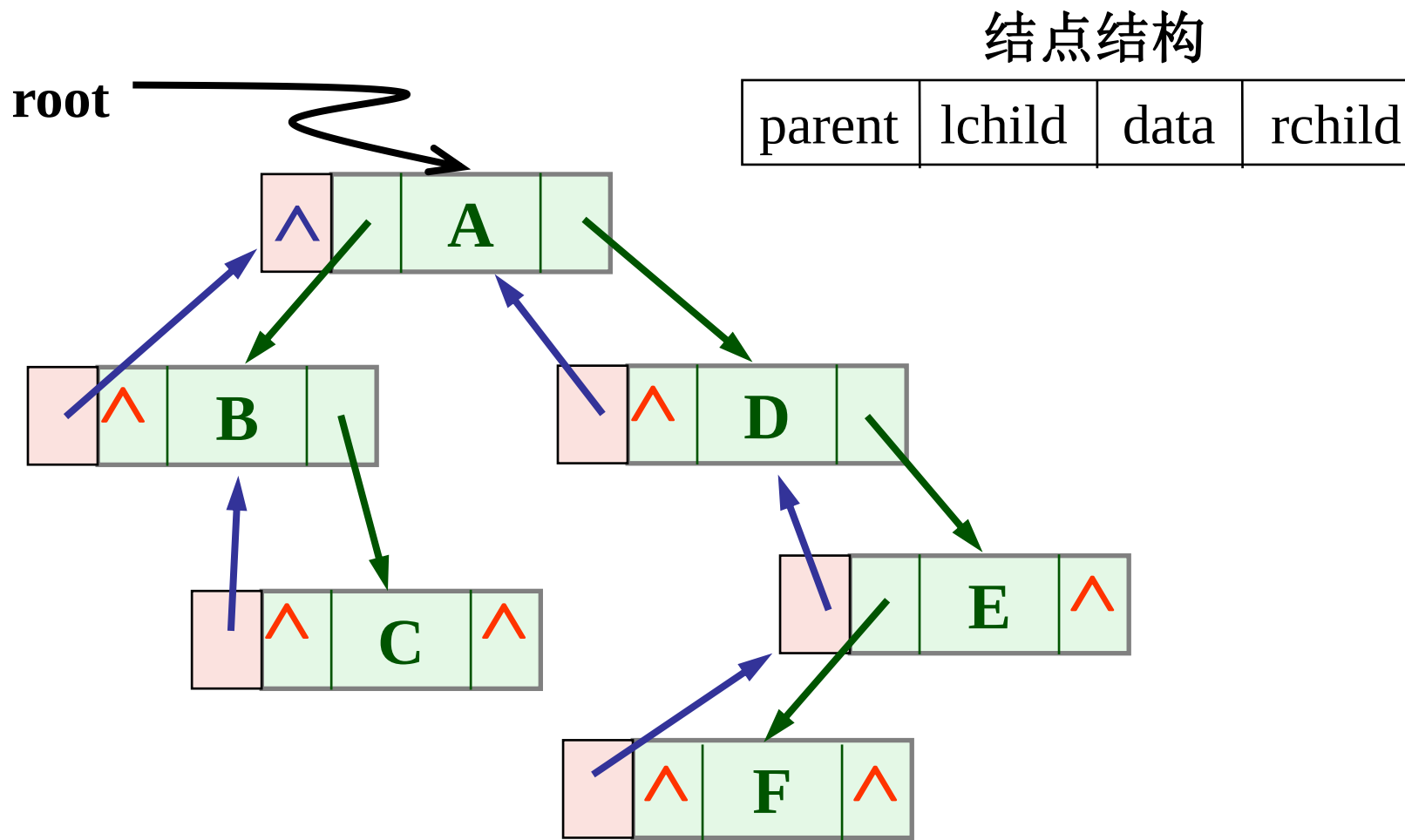


三叉链表

二、链式存储表示-二叉链表



二、链式存储表示-三叉链表



- 6.1 树的基本概念
- 6.2 二叉树的定义、性质及运算
- 6.3 二叉树的存储结构
- 6.4 遍历二叉树
- 6.5 线索二叉树
- 6.6 树和森林
- 6.7 哈夫曼树、哈夫曼编码

遍历的概念

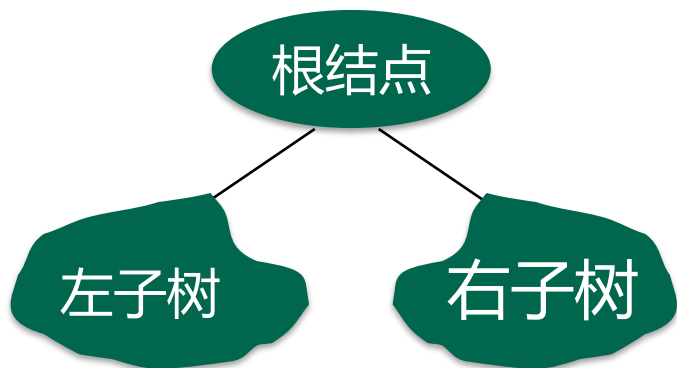
从根节点出发，**按照某种次序**，依次访问二叉树中的所有结点，使得每个结点**均被访问一次**，而且**仅被访问一次**。

“访问”的含义很广，可以是对结点作各种处理，如：输出结点的信息、修改结点的数据值等，但要求这种访问不破坏原来的数据结构。

遍历目的

得到树中所有结点的一个线性排列。

遍历方法



依次遍历二叉树中的三个组成部分，便是遍历了整个二叉树

假设：L：遍历左子树，**V**：访问根结点，R：遍历右子树，
则遍历整个二叉树方案共有：

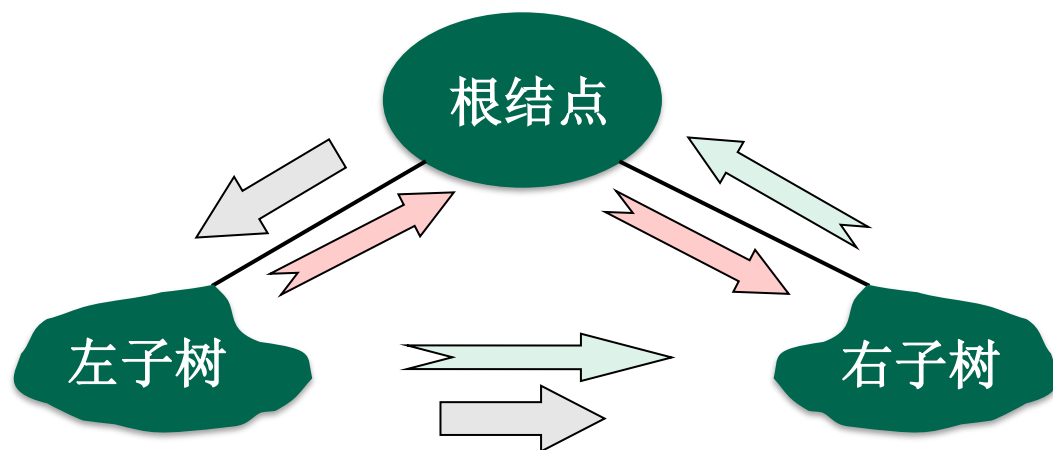
VLR、L**V**R、LR**V**、**V**RL、R**V**L、RL**V** 六种。

若规定先左后右，则只有前三种情况：

VLR —— 先（**根**）序遍历，

L**V**R —— 中（**根**）序遍历，

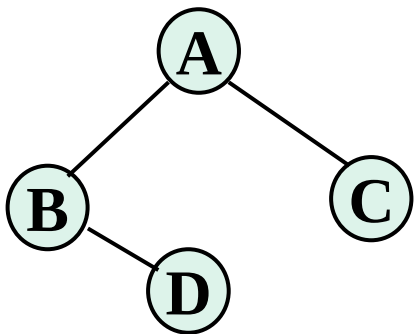
LR**V** —— 后（**根**）序遍历。



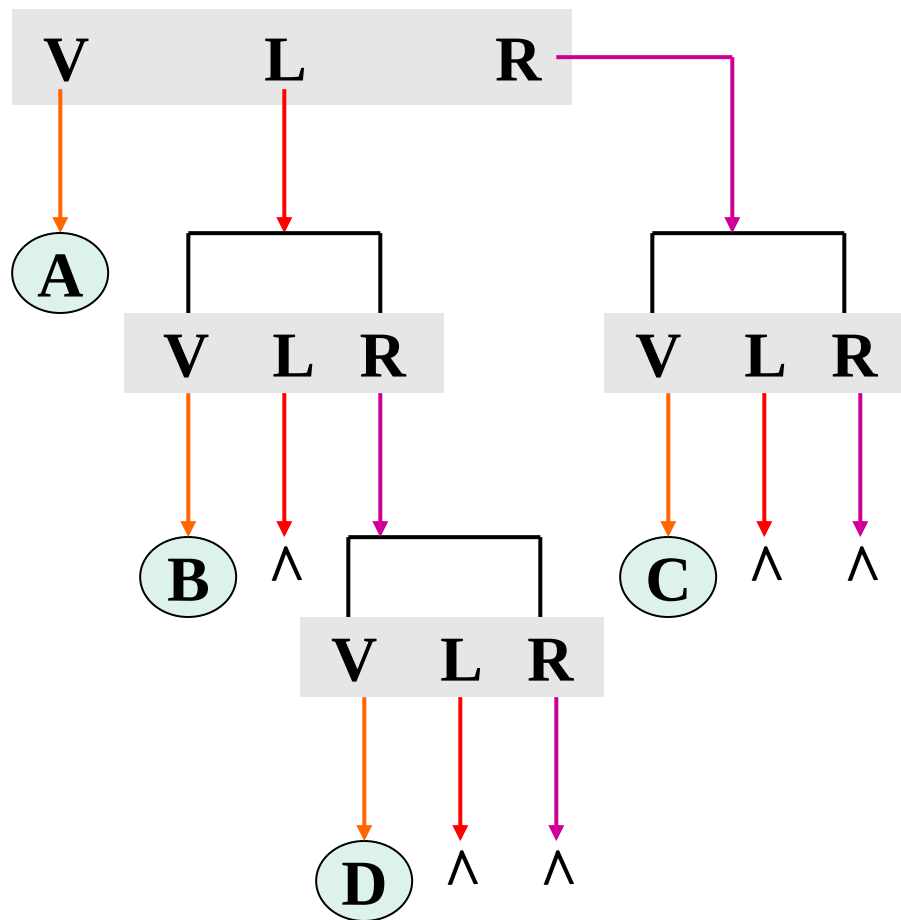
先序遍历 (VLR) 二叉树的操作定义：

若二叉树为空，则空操作；否则

- (1) 访问根结点；
- (2) 先序遍历左子树；
- (3) 先序遍历右子树。



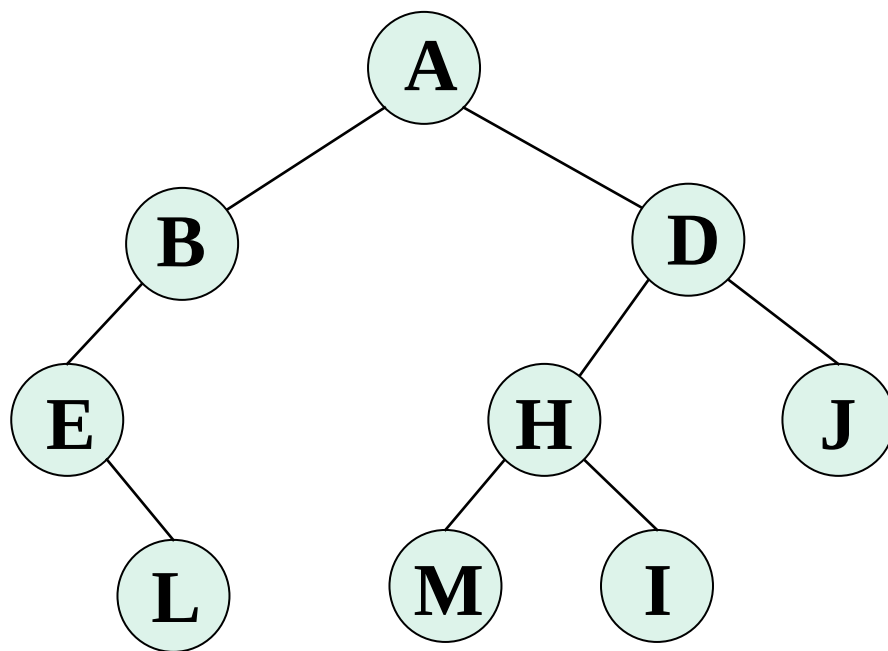
先序遍历序列：A B D C



先序遍历 (VLR) 二叉树的操作定义：

若二叉树为空，则空操作；否则

- (1) 访问根结点；
- (2) 先序遍历左子树；
- (3) 先序遍历右子树。

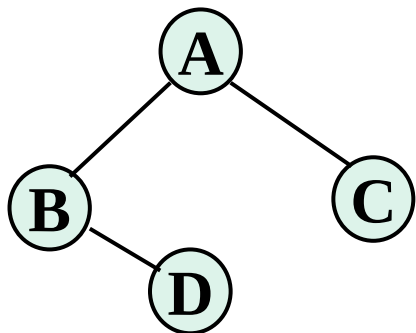


先序遍历结果：A B E L D H M I J

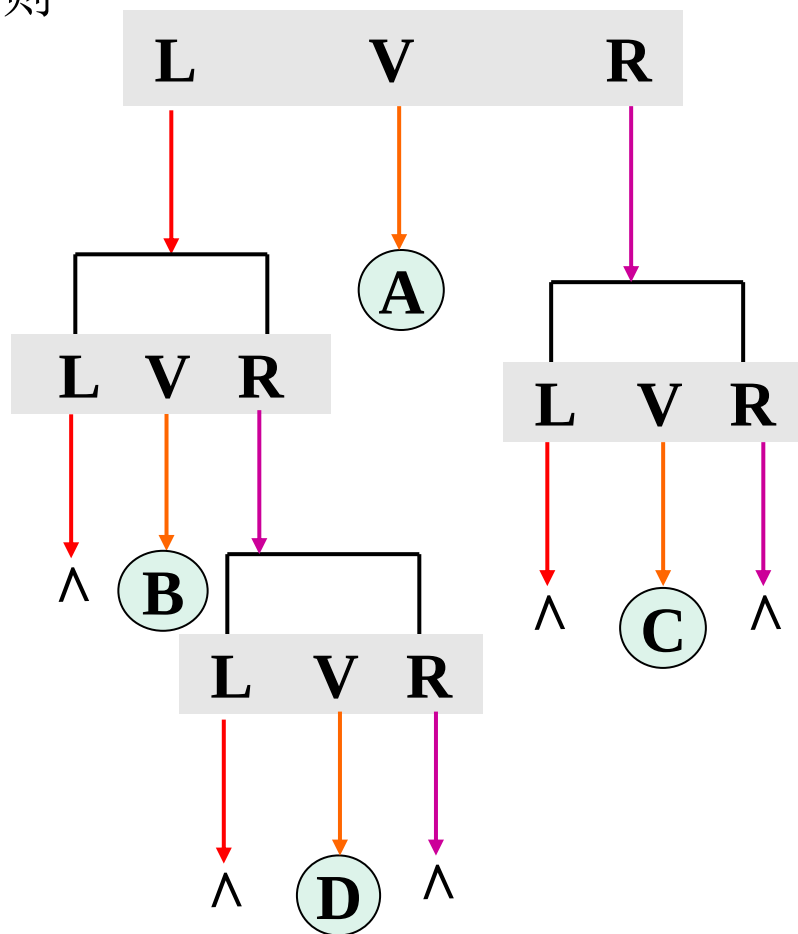
中序遍历(LVR)二叉树的操作定义：

若二叉树为空，则空操作；否则

- (1) 中序遍历左子树；
- (2) 访问根结点；
- (3) 中序遍历右子树。



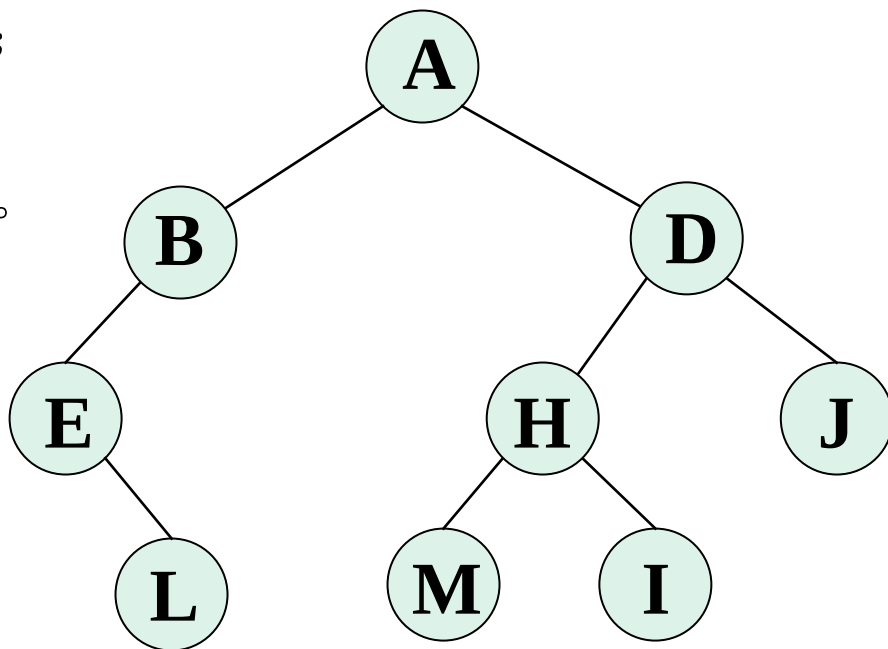
中序遍历序列：B D A C



中序遍历(LVR)二叉树的操作定义：

若二叉树为空，则空操作；否则

- (1) 中序遍历左子树；
- (2) 访问根结点；
- (3) 中序遍历右子树。

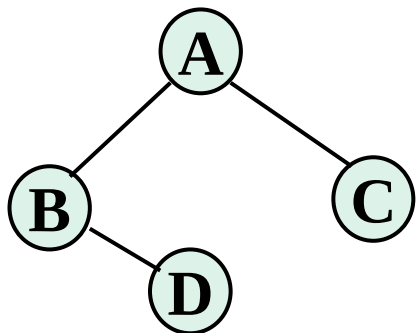


中序遍历结果：E L B A M H I D J

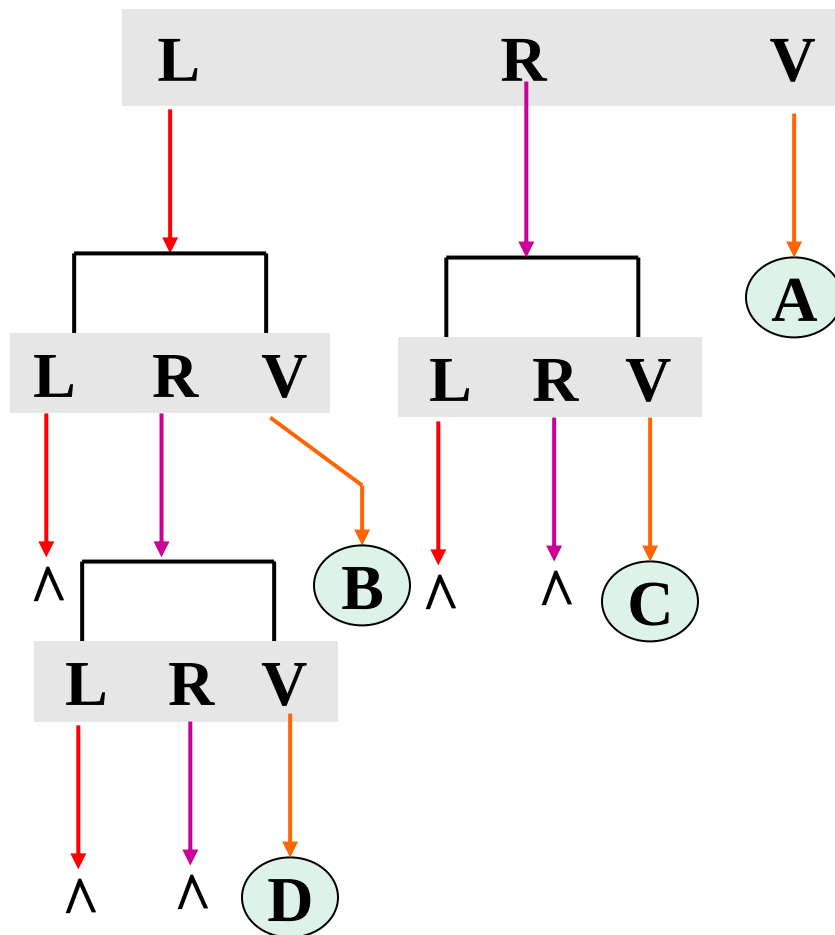
后序遍历(LRV)二叉树的操作定义：

若二叉树为空，则空操作；否则

- (1) 后序遍历左子树；
- (2) 后序遍历右子树；
- (3) 访问根结点。



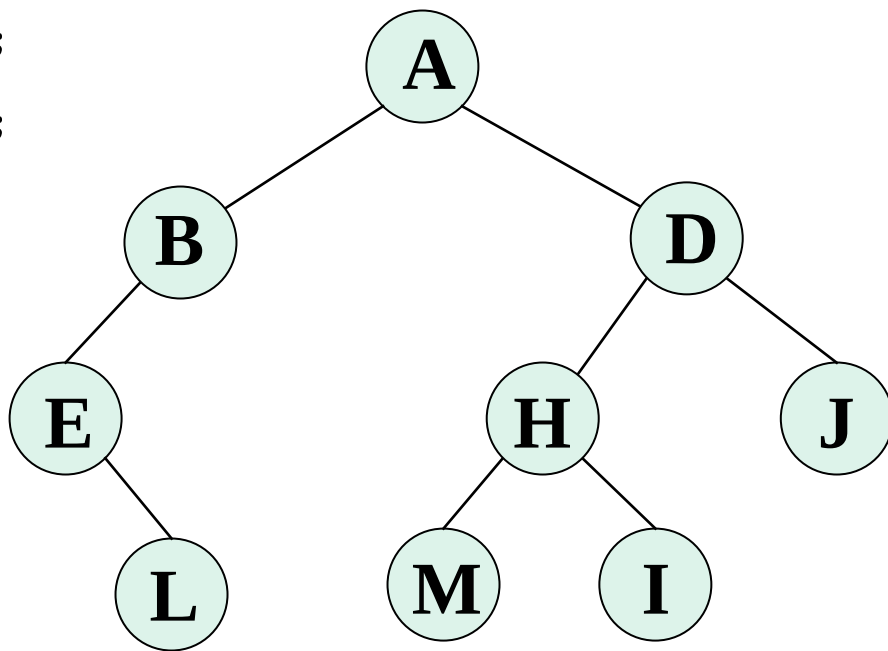
后序遍历序列：D B C A



后序遍历(LR $\mathbf{V}$)二叉树的操作定义:

若二叉树为空，则空操作；否则

- (1) 后序遍历左子树；
- (2) 后序遍历右子树；
- (3) 访问根结点。



中序遍历结果: L E B M I H J D A

表达式和二叉树的关系

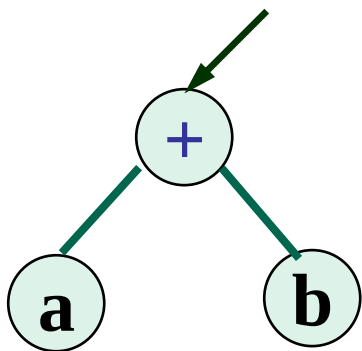
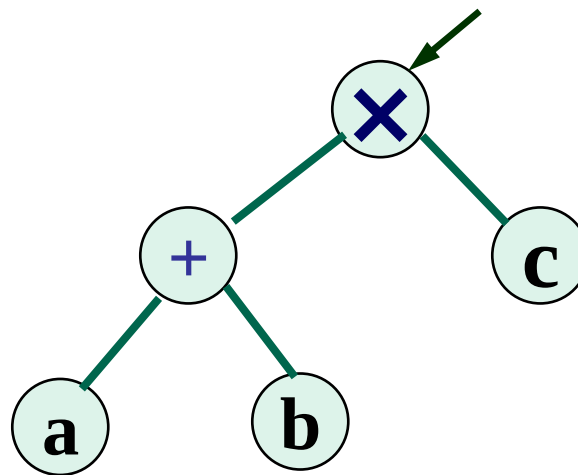
以二叉树表示表达式的递归定义如下：

若表达式为一个数（或简单变量），则对应的二叉树中仅有一个根结点，其数据域存放该表达式的信息；

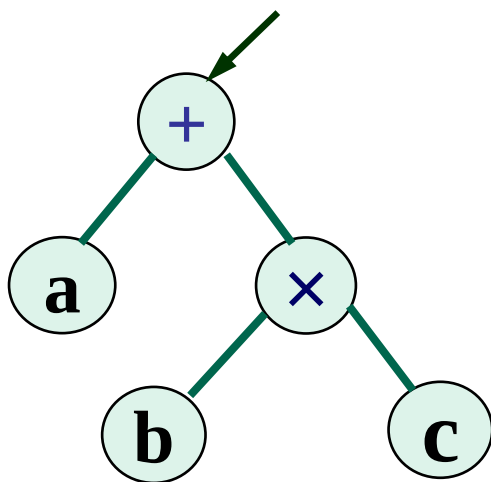
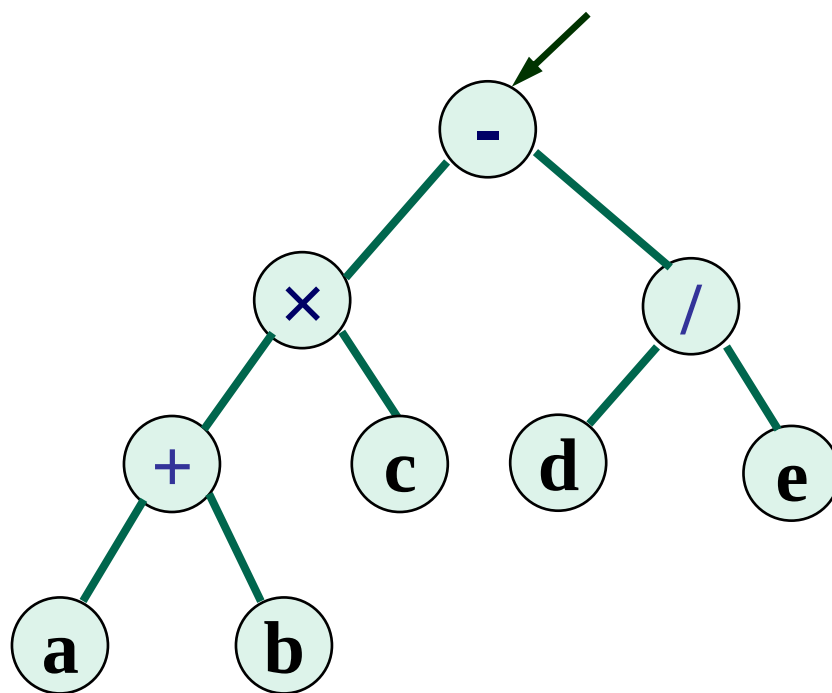
若表达式形如“（第一操作数）（运算符）（第二操作数）”，则对应的二叉树以左子树表示第一操作数，右子树表示第二操作数，根结点的数据域存放运算符（若为一元运算符，则左子树为空）。

操作数本身又为表达式。

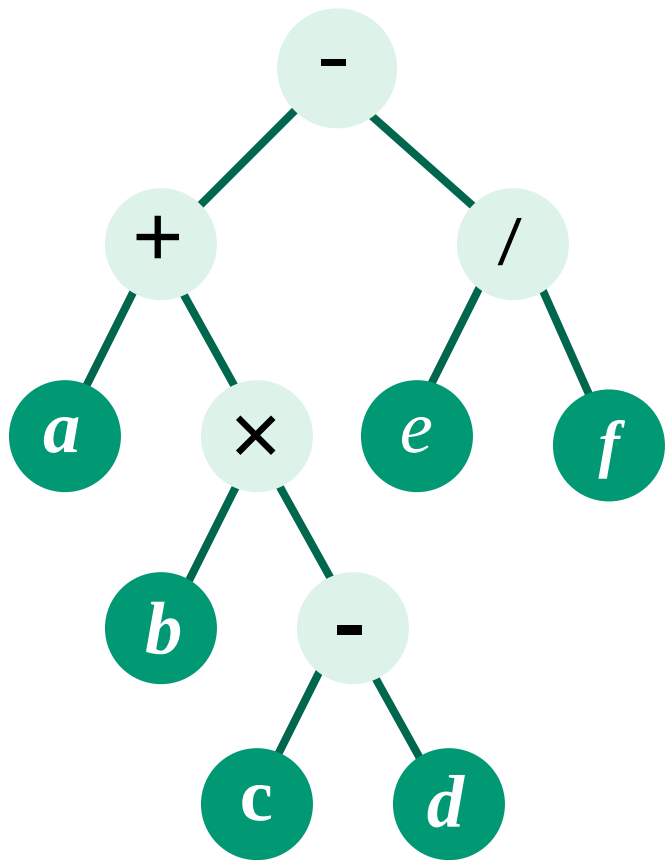
表达式和二叉树的关系

 $a+b$  $(a+b) \times c$ 

表达式和二叉树的关系

 $a+b\times c$  $(a+b)\times c - d/e$ 

例：写出下图二叉树的先序、中序和后序遍历顺序。



遍历结果：

前序 (VLR): $- + a \times b - c d / e f$

表达式的前缀表示

中序 (LVR): $a + b \times c - d - e / f$

表达式的中缀表示

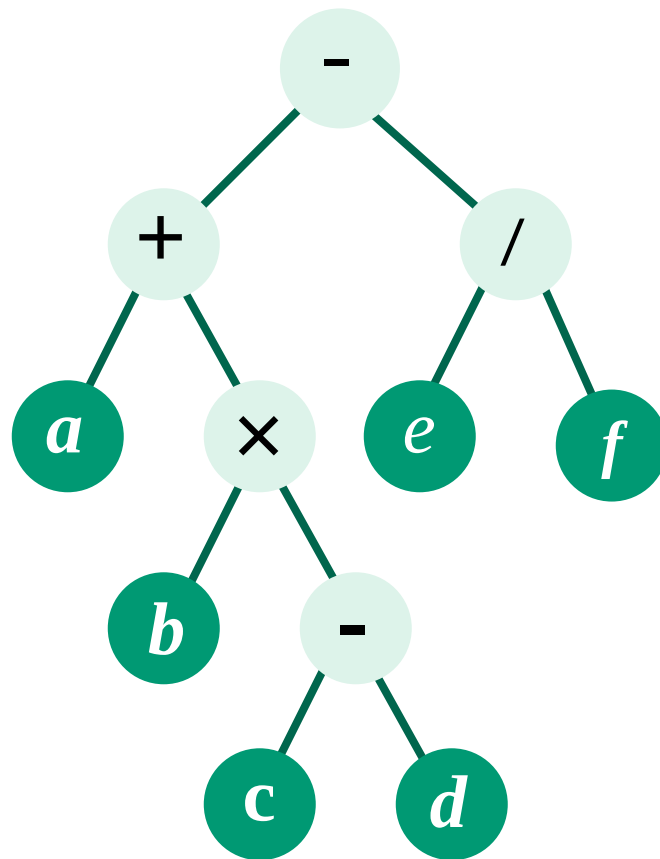
后序 (LRV): $a b c d - \times + e f / -$

表达式的后缀表示

层次遍历(Levelorder Traversal)

若树为空，则空操作返回，否则
从树的第一层，即根节点开始
从上到下逐层访问
在同一层，从左到右逐节点访问

遍历结果 **- + / a × e f b - c d**



结点数据类型自定义

```
typedef struct node{  
    int data;  
    struct node *lchild,*rchild ;  
} node;  
node *root;
```

中序遍历算法

```
LVR(node *root)  
{if(root !=NULL)  
    {LVR(root->lchild);  
    printf(“%d”,root->data);  
    LVR(root->rchild);  
    } return(0);}
```

先序遍历算法

```
VLR( node *root )  
{if (root !=NULL) //非空二叉树  
    {printf(“%d”,root->data); //访问D  
    VLR(root->lchild); //递归遍历左子树  
    VLR(root->rchild); //递归遍历右子树  
    } return(0); }
```

后序遍历算法

```
LRV (node *root)  
{if(root !=NULL)  
    {LRV(root->lchild);  
    LRV(root->rchild);  
    printf(“%d”,root->data);  
    } return(0);}
```

非递归的二叉树前序遍历

非递归算法，引入栈模拟递归过程，初始时栈为空。

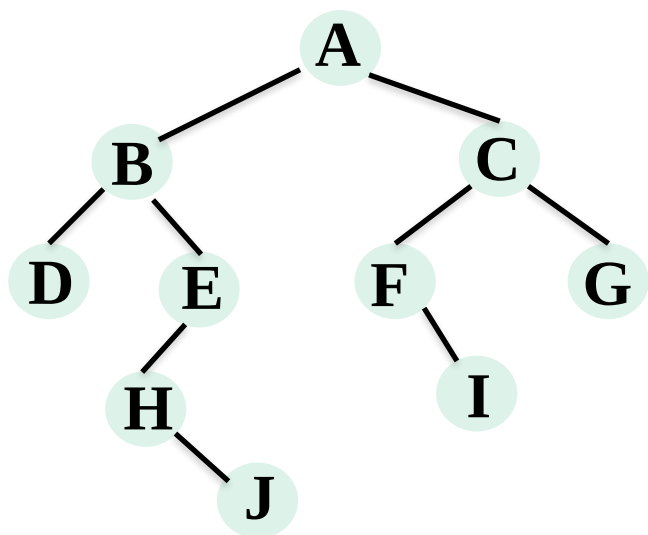
问题是如何利用栈来保存信息，使得在前序遍历节点 t 的左子树后，能利用栈顶信息获取节点 t 的右子树的根指针？

方法是访问 $t \rightarrow data$ 后，将 t 入栈，遍历左子树；遍历完左子树返回时，栈顶元素应为 t ，出栈，再先序遍历 t 的右子树。

非递归的二叉树前序遍历

- 1) 建立栈**stack**, 初始时栈为空
- 2) t 指向根
- 3) 当 t 不为空时, 或栈**stack**不空时, 重复:
 - 若 t 不空, 访问 $t \rightarrow data$ 后, 将 t 入栈;
 - t 指向其左孩子;
 - 否则: 栈顶元素出栈;
 - t 指向其右孩子
- 4) 结束

非递归的二叉树前序遍历



把根存入栈中

访问A, A进栈, t指向B	栈[A]
访问B, B进栈 , t指向D	栈[AB]
访问D, D进栈 , t为空	栈[ABD]
D出栈, t为空	栈[AB]
B出栈, t指向E	栈[A]
访问E, E进栈 , t指向H	栈[AE]
访问H, H进栈 , t为空	栈[AEH]
H出栈, t指向J	栈[AE]
访问J, J进栈 , t为空	栈[AEJ]
J出栈, t为空	栈[AE]
E出栈, t为空	栈[A]
A出栈 , t指向C	栈为空
访问C, C进栈, t指向F	栈[C]
访问F, F进栈, t为空	栈[CF]
F出栈, t指向I	栈[C]
访问I, I进栈, t为空	栈[CI]
I出栈, t为空	栈[C]
C出栈, t指向G	栈为空
访问G, G进栈, t为空	栈[G]
G出栈, t为空	栈为空

非递归的二叉树前序遍历

// 二叉树节点定义

```
typedef struct node {  
    datatype data;  
    struct node *lchild, *rchild  
}bintnode;  
typedef bintnode *bintree;
```

// 进栈函数

```
void push(seqstack *s, bintree t) {  
    s→data[s→top++]=t;  
}
```

// 栈结构定义

```
typedef struct stack {  
    bintree data[100];  
    int top; //栈顶指针  
}seqstack;
```

// 出栈函数

```
bintree pop(seqstack *s) {  
    if (s→top != -1) {  
        return (s→data[--s→top]);  
    } else  
        return NULL;  
}
```

非递归的二叉树前序遍历

// 非递归实现二叉树的前序遍历

```
void preorder1(bintree t) {  
    seqstack s;  
    s.top = -1;  
    // 当前处理的子树不为空或栈不为空则循环  
    while ( (t) || (s.top != -1) ) {  
        if (t) {  
            printf("%f", t->data);  
            s.data[s.top++] = t;  
            t = t->lchild;  
        } // endwhile  
        else {  
            t = pop(&s);  
            t = t->rchild;  
        } // endifelse  
    } // endwhile  
} // end preorder1
```

二叉树的重建

如何由结点的线性排列重建二叉树？

由二叉树的前序序列和中序序列**可以**唯一地确定一棵二叉树

由二叉树的后序序列和中序序列**可以**唯一地确定一棵二叉树

由二叉树的前序序列和后序序列**不可**唯一地确定一棵二叉树

由二叉树的前序和中序序列建树

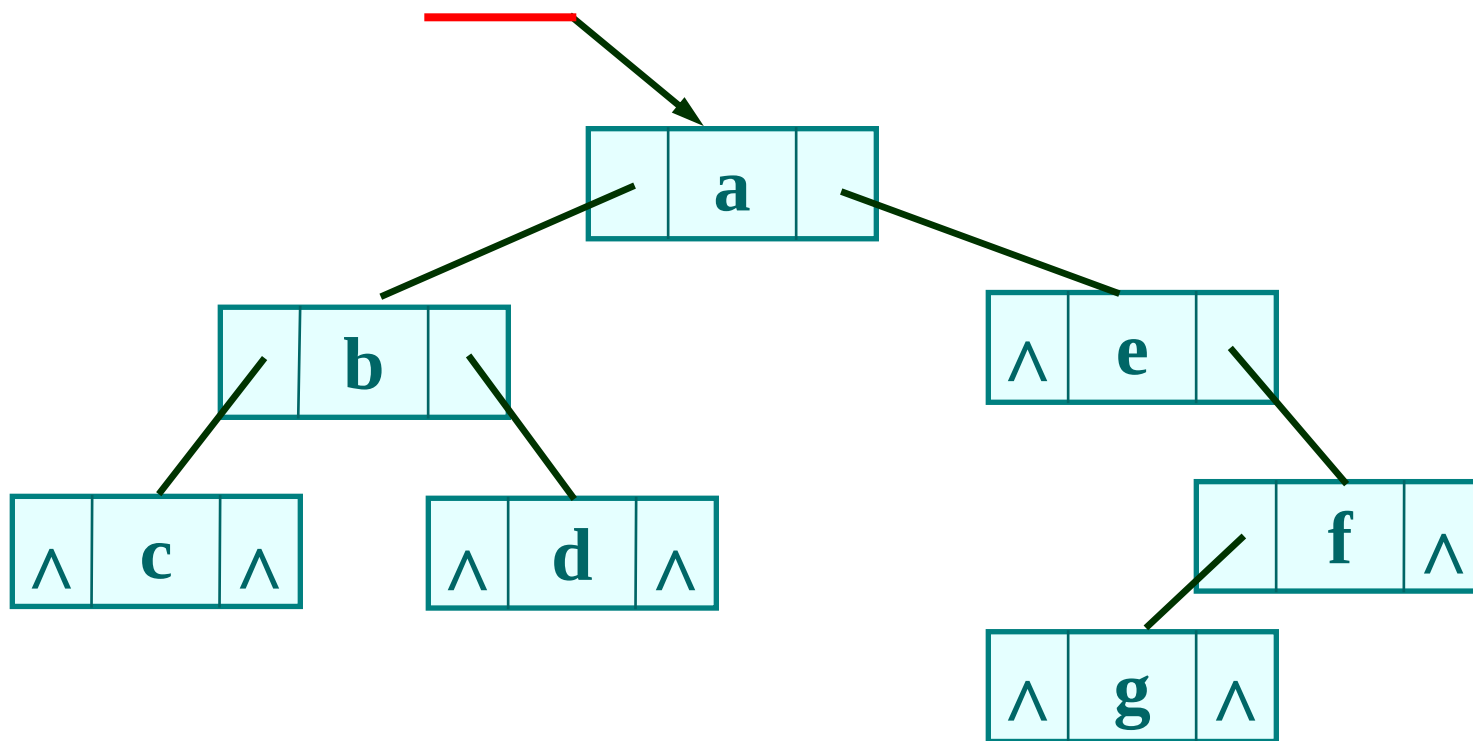
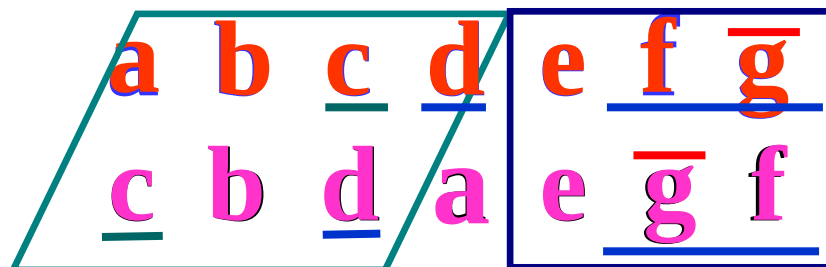
仅知二叉树的前序序列“abcdefg”不能唯一确定一棵二叉树，如果同时已知二叉树的中序序列“cbdaegf”，则会如何？

二叉树的前序序列



二叉树的中序序列

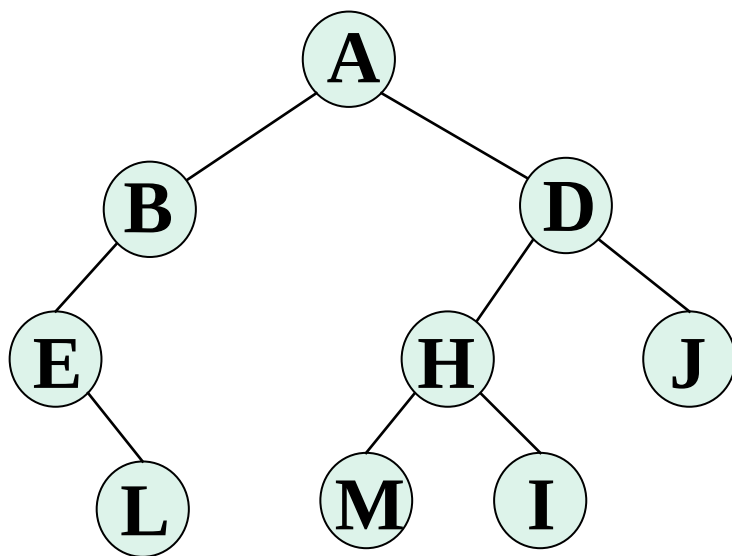




- 6.1 树的基本概念
- 6.2 二叉树的定义、性质及运算
- 6.3 二叉树的存储结构
- 6.4 遍历二叉树
- 6.5 线索二叉树
- 6.6 树和森林
- 6.7 哈夫曼树、哈夫曼编码

遍历二叉树的目的：

对非线性化结构进行线性化，每个节点，在这个线性化序列中有且只有一个直接前驱和一个直接后继。



如何保存前驱和后继的信息？

指向前驱和后继的指针称为**线索**

带**线索**的二叉树称为**线索二叉树**

中序遍历结果：L E B M I H J D A

6.1 树的基本概念

6.2 二叉树的定义、性质及运算

6.3 二叉树的存储结构

6.4 遍历二叉树

6.5 线索二叉树

6.6 树和森林

6.7 哈夫曼树、哈夫曼编码

树和森林与二叉树的转换

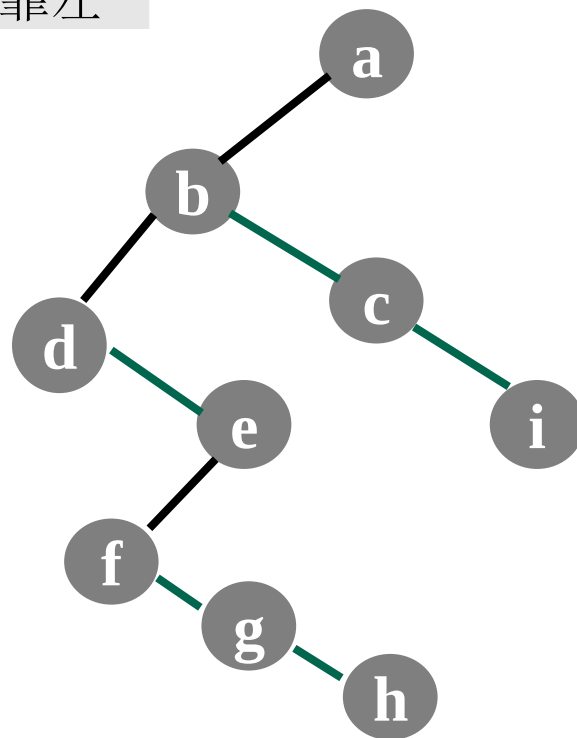
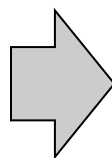
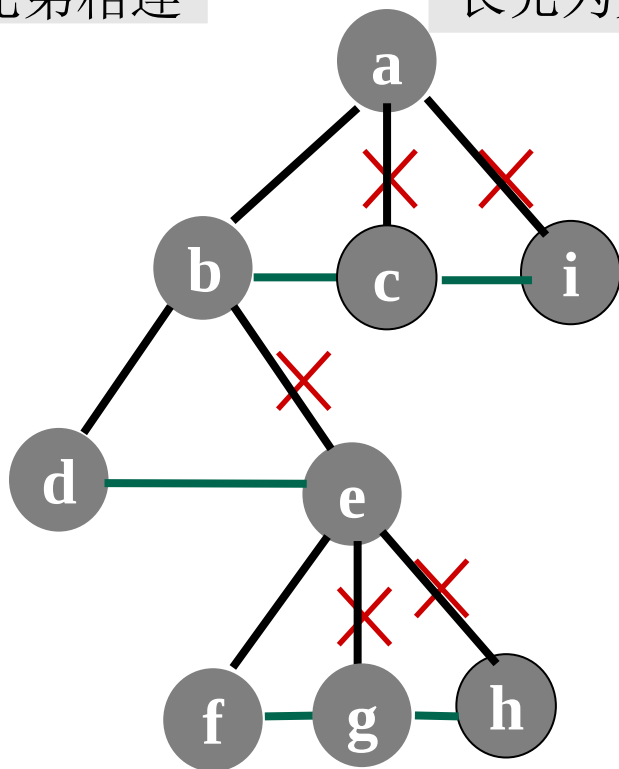
回顾1：树如何转为二叉树？

方法：加线—抹线—旋转

兄弟相连

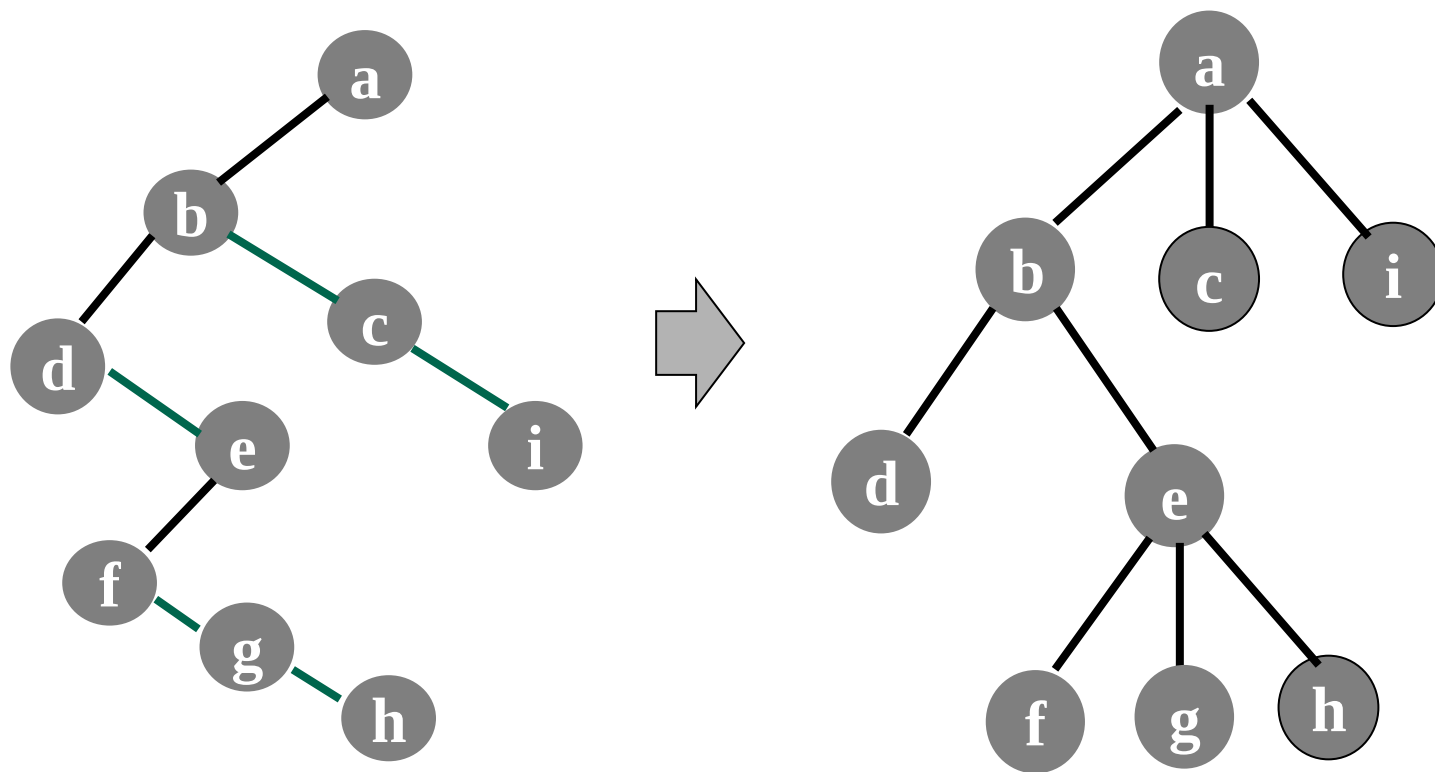
长兄为父

孩子靠左



树和森林与二叉树的转换 回顾2：二叉树怎样还原为树？

要点：逆操作，把所有右孩子变为兄弟！



讨论1：森林如何转为二叉树？

即 $F = \{T_1, T_2, \dots, T_m\} \longleftrightarrow B = \{\text{root}, \text{LB}, \text{RB}\}$

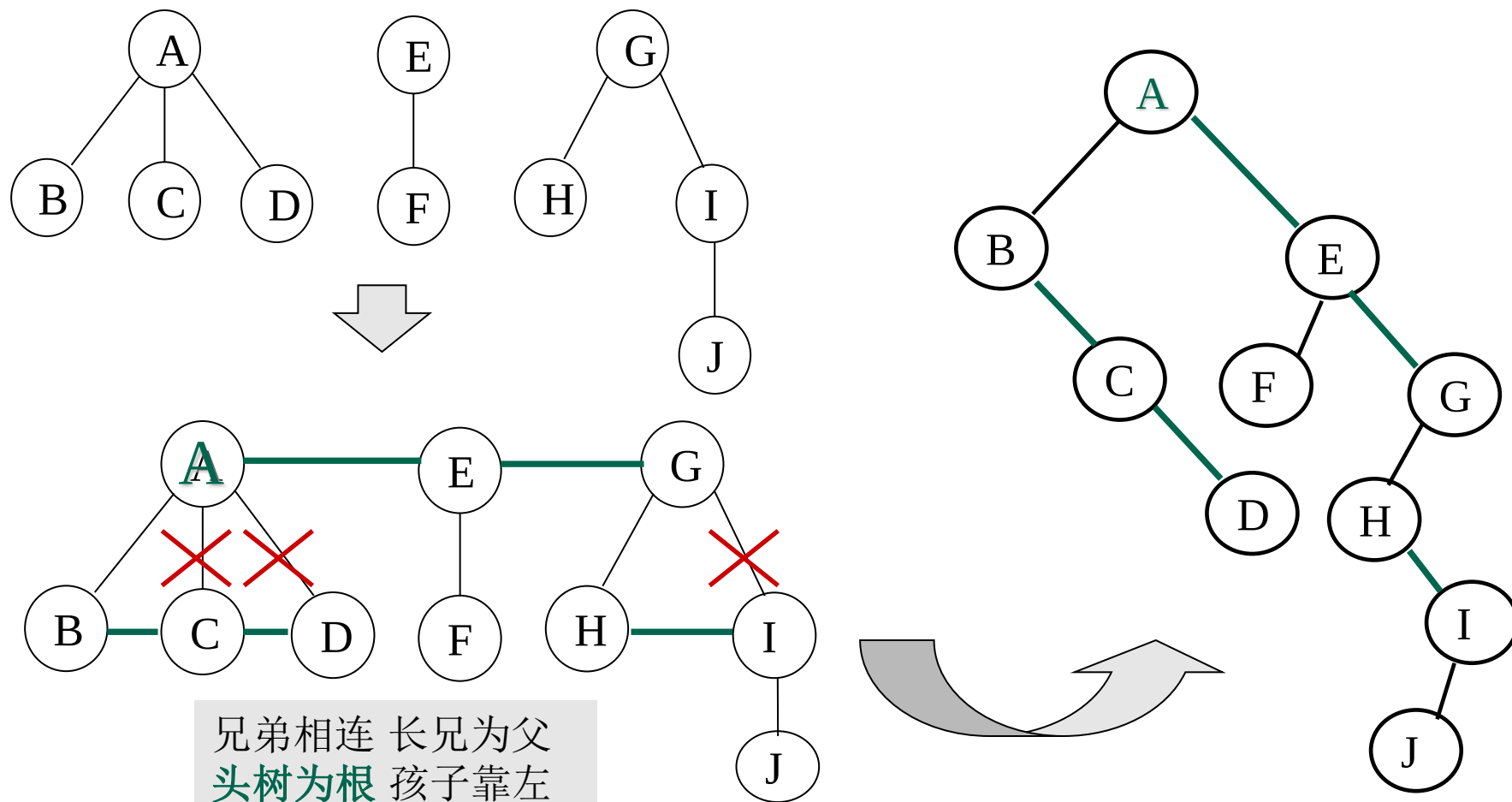
法一：① 各森林先各自转为二叉树；
② 依次连到前一个二叉树的右子树上。

法二：森林直接变兄弟，再转为二叉树

法一和法二得到的二叉树是完全相同的、惟一的。

森林转二叉树举例：

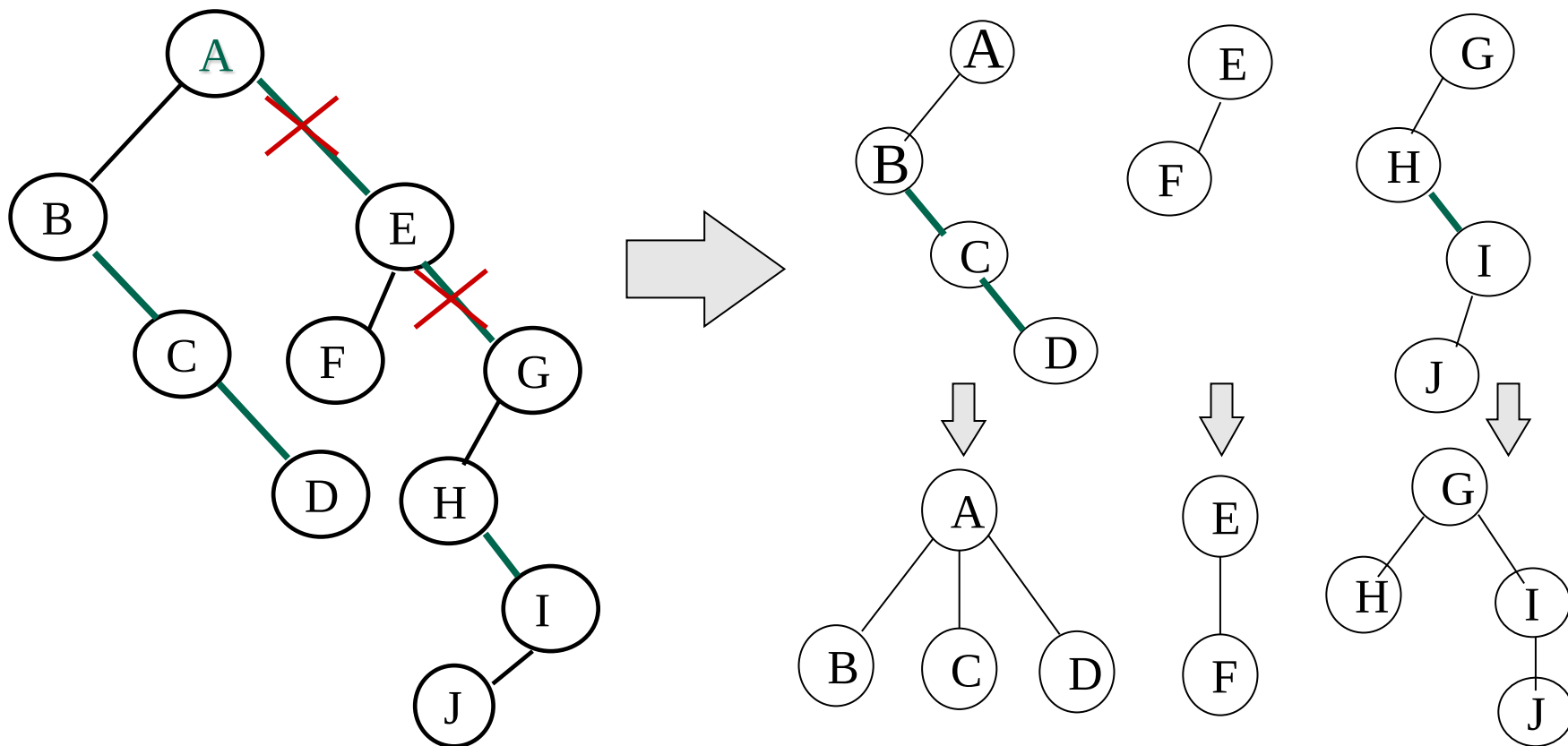
（用法二，森林直接变兄弟，再转为二叉树）



讨论2：二叉树如何还原为森林？

即 $B = \{\text{root}, \text{LB}, \text{RB}\} \longleftrightarrow F = \{T_1, T_2, \dots, T_m\}$

要点：把最右边的子树变为森林，其余右子树变为兄弟



树和森林的存储方式

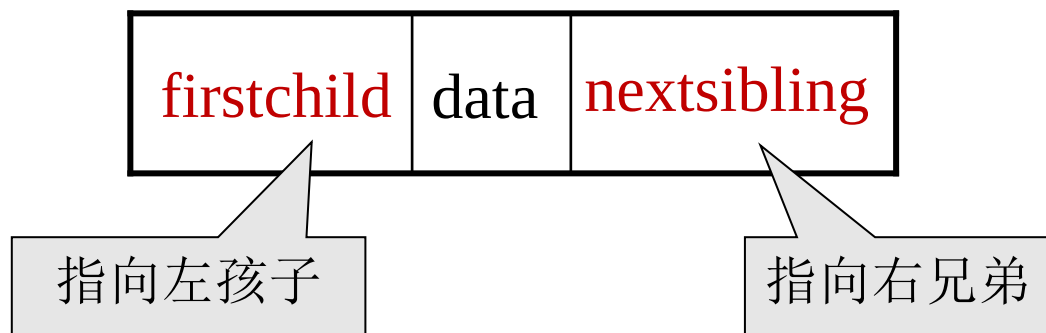
树有三种常用存储方式：

①双亲表示法 ②孩子表示法 ③孩子—兄弟表示法

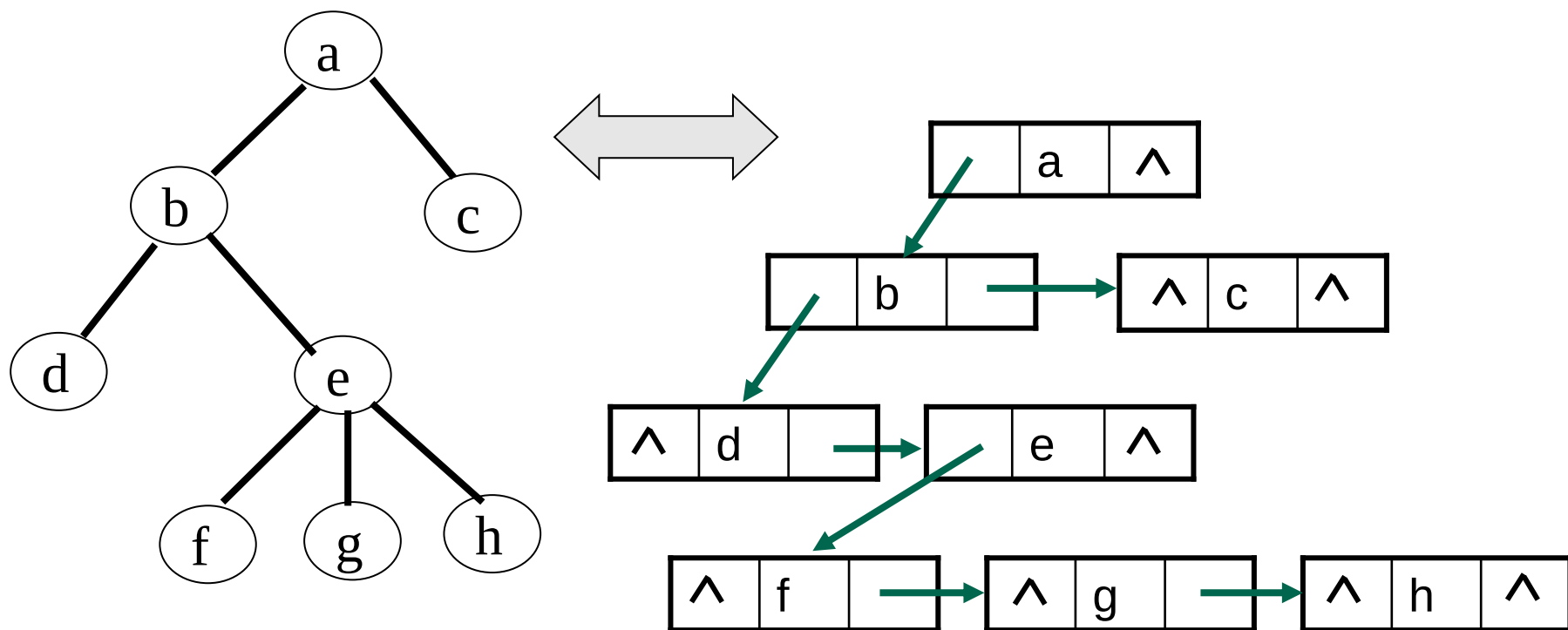
问：树→二叉树的“连线—抹线—旋转”如何由计算机自动实现？

答：用“左孩子右兄弟”表示法来存储即可。

存储的过程就是树转换为二叉树的过程！



例如：



树和森林的遍历

树的遍历 { 深度优先遍历（先根、后根）
 广度优先遍历（层次）

树没有中序遍历
（因子树不分左右）

先根遍历

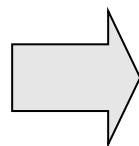
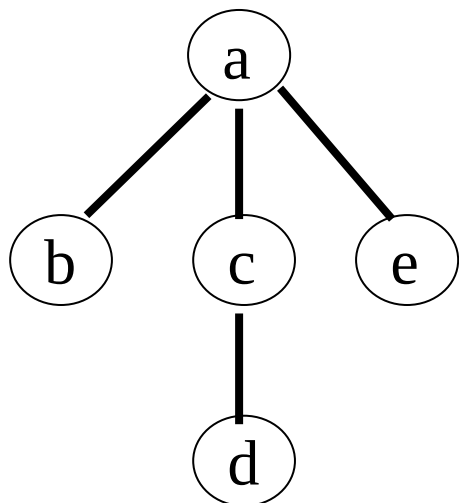
访问根结点；
依次先根遍历根结点的每棵子树。

后根遍历

依次后根遍历根结点的每棵子树；
访问根结点。

树和森林的遍历

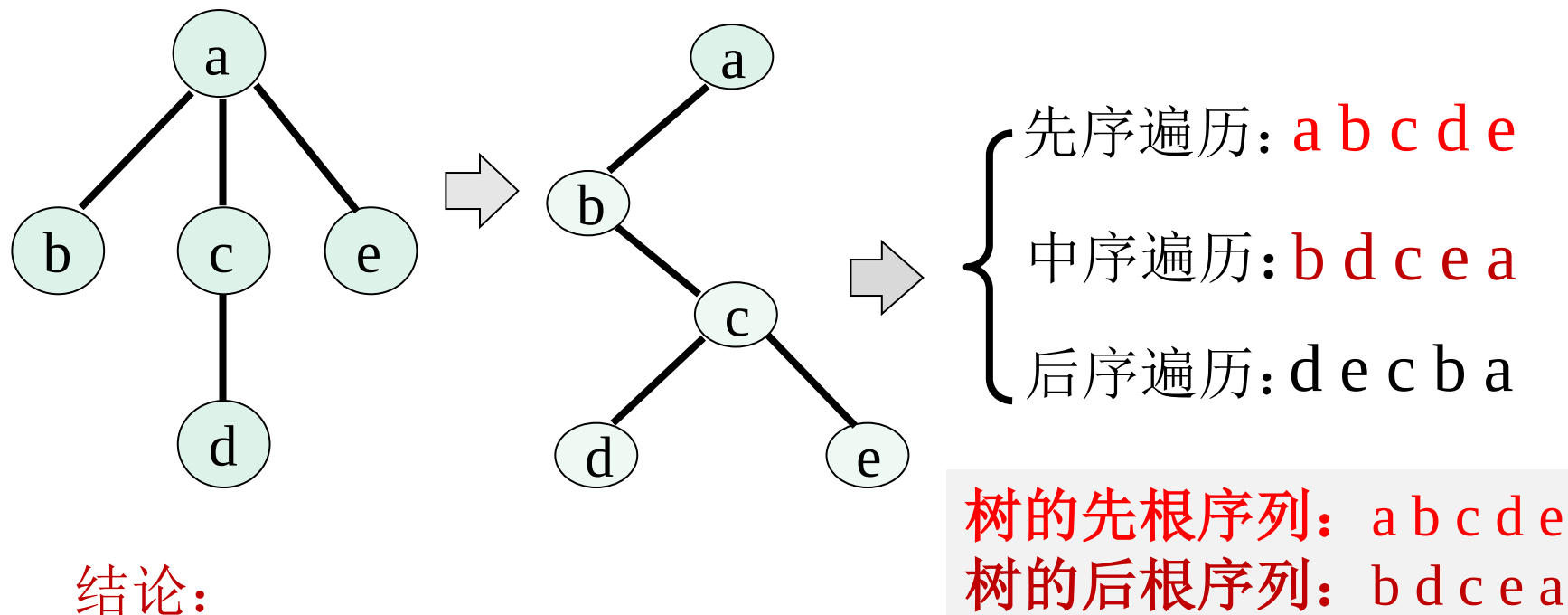
例如：



先根序列： a b c d e

后根序列： b d c e a

讨论：树若采用“先转换，后遍历”方式，结果是否一样？



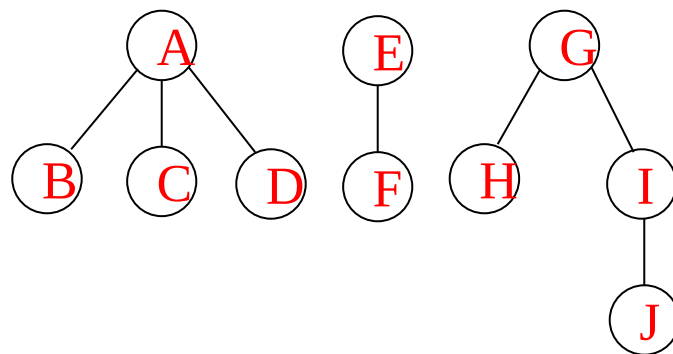
结论：

1. 树的**先根**遍历与二叉树的**先序**遍历相同；
2. 树的**后根**遍历相当于二叉树的**中序**遍历；
3. 树没有中序遍历，因为子树无左右之分。

森林的遍历 { 深度优先遍历（先序、中序）
 广度优先遍历（层次）

先序遍历

- ① 若森林为空，返回；
- ② 访问森林中第一棵树的根结点；
- ③ 先序遍历第一棵树的根结点的子树森林；
- ④ 先序遍历除去第一棵树之后剩余的树构成的森林。

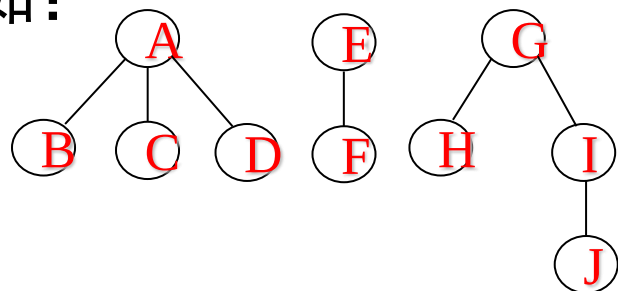


森林的遍历 { 深度优先遍历（先序、中序）
 { 广度优先遍历（层次）

中序遍历

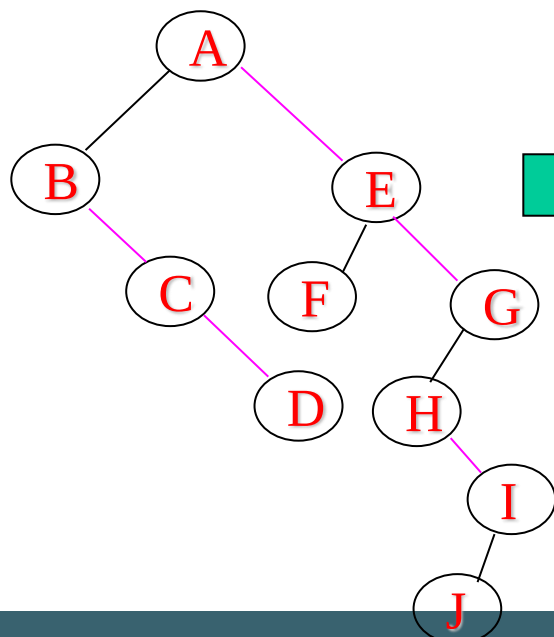
- 若森林为空，返回；
- 中序遍历森林中第一棵树的根结点的子树森林；
- 访问第一棵树的根结点；
- 中序遍历除去第一棵树之后剩余的树构成的森林。

例如：



先序序列： **A B C D E F G H I J**
中序序列： **B C D A F E H J I G**

讨论：若采用“先转换，后遍历”方式，结果是否相同？



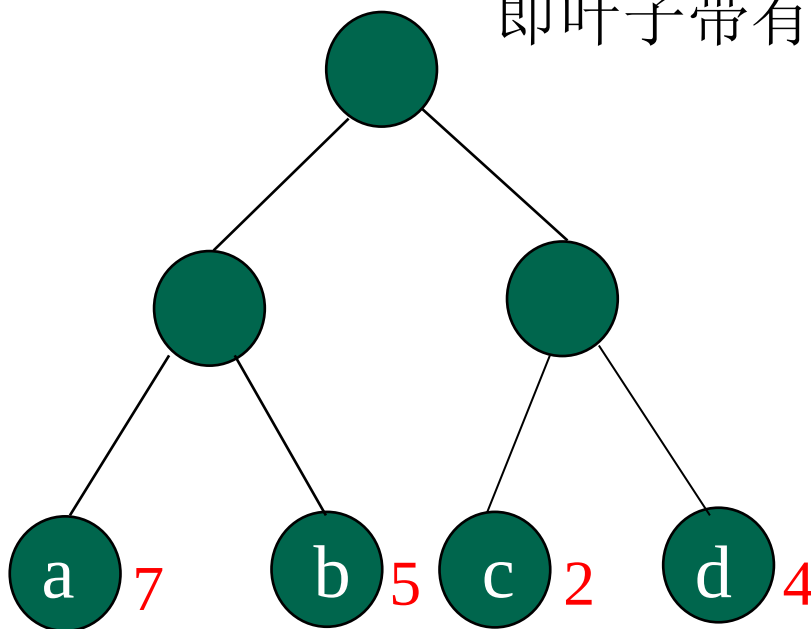
先序序列： **A B C D E F G H I J**
中序序列： **B C D A F E H J I G**

结论：森林的先序和中序遍历在两种方式下的结果相同。

- 6.1 树的基本概念
- 6.2 二叉树的定义、性质及运算
- 6.3 二叉树的存储结构
- 6.4 遍历二叉树
- 6.5 线索二叉树
- 6.6 树和森林
- 6.7 哈夫曼树、哈夫曼编码

什么是带权树？

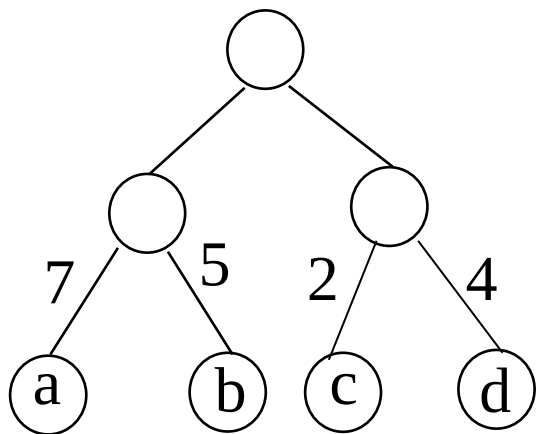
即叶子带有权值。例如：



树的带权路径长度如何计算？

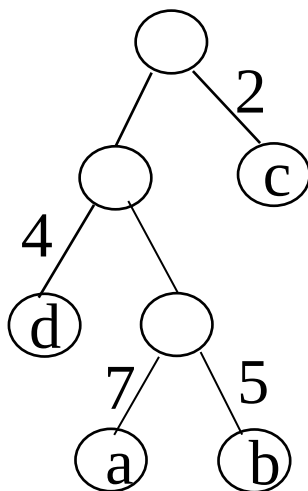
$$WPL = \sum_{k=1}^n w_k l_k$$

树中所有叶子结点的带权路径长度之和



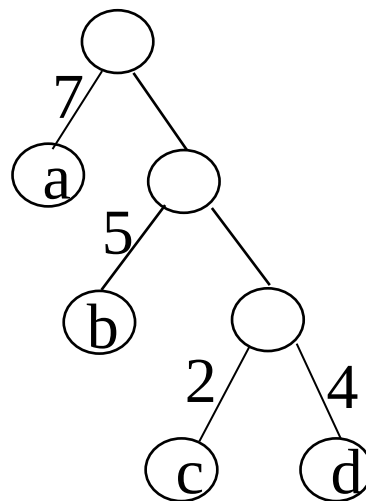
(a)

WPL=36



(b)

WPL=46



(c)

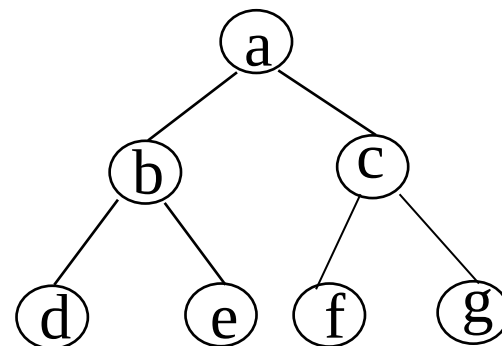
WPL=35

哈夫曼树是 WPL 最小的树

一、哈夫曼树（最优二叉树）

构造哈夫曼树的基本思想：

- 权值大的结点用短路径，
- 权值小的结点用长路径。



若干术语：

路 径：由一结点到另一结点间的分支所构成。

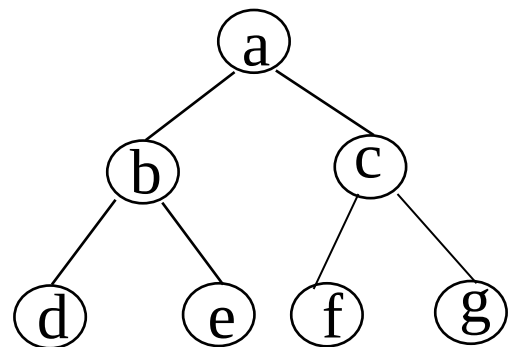
路径长度：路径上的分支数目例如： $a \rightarrow e$ 的路径长度=2

树的路径长度：从树根到每一结点的路径长度之和。

树长度=10

一、哈夫曼树（最优二叉树）

若干术语：



带权路径长度：结点到根的路径长度与结点上权的乘积（WPL）

Weighted Path Length

树的带权路径长度：即树中所有叶子结点的带权路径长度之和

哈夫曼树：带权路径长度最小的树。

也称为赫夫曼、霍夫曼等

二、构造哈夫曼树的步骤（哈夫曼算法）

- (1) 由给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构成 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$ （即森林），其中每棵二叉树 T_i 中只有一个带权为 w_i 的根结点，其左右子树均空。
- (2) 在 F 中选取**两棵根结点权值最小的树**做为左右子树构造一棵新的二叉树，且让新二叉树根结点的权值**等于**其左右子树的根结点权值之和。
- (3) 在 F 中删去这两棵树，同时将新得到的二叉树加入 F 中。
- (4) 重复(2)和(3)，直到 F 只含一棵树为止。这棵树便是哈夫曼树。

二、构造哈夫曼树的步骤（哈夫曼算法）

对权值进行合并、删除与替换

——在权值集合{7,5,2,4}中，总是合并当前值最小的两个权

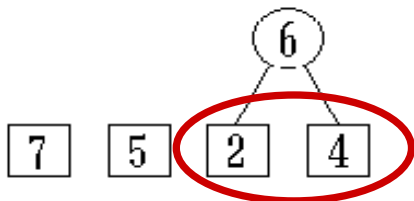
a. 初始

F: {7,5,2,4}



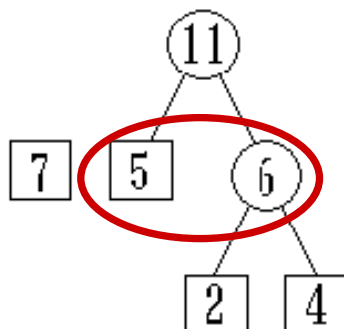
b. 合并{2} {4}

F: {7,5,6}



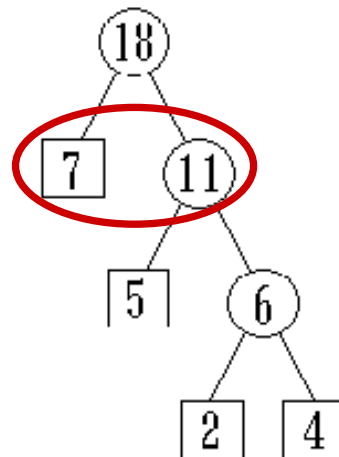
c. 合并{5} {6}

F: {7,11}



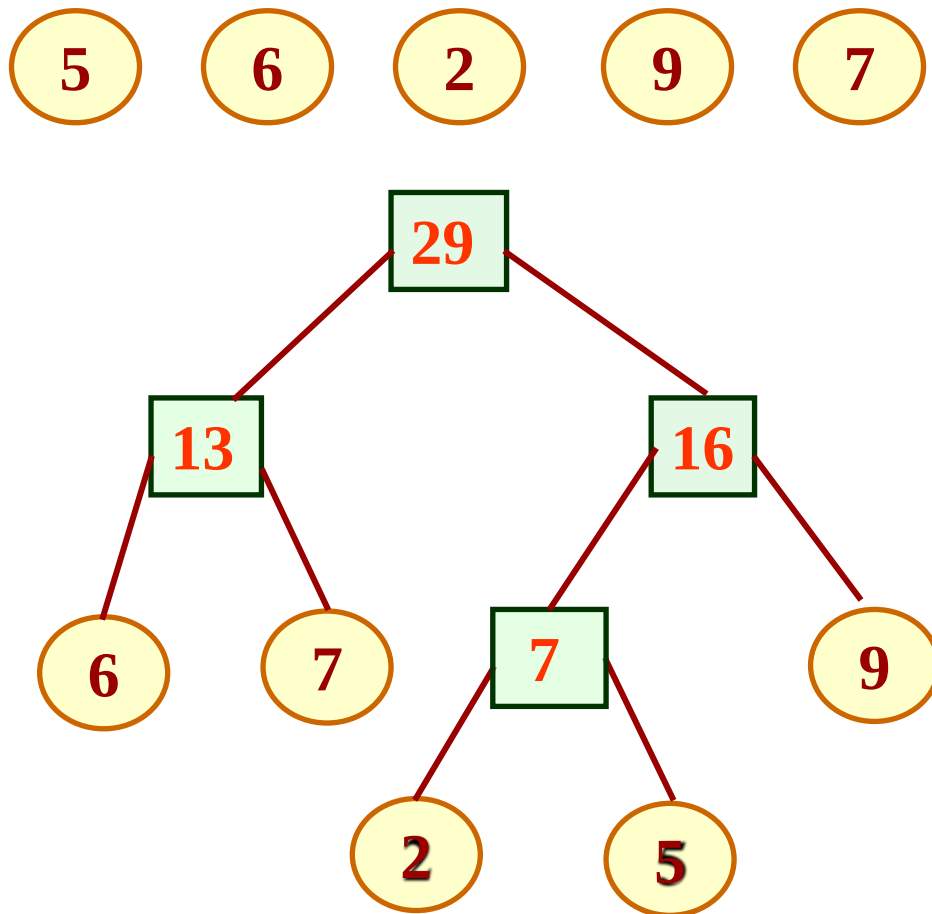
d. 合并{7} {11}

F: {18}



谁左谁右?
不规定就不会惟一

例题：已知权值 $W=\{ 5, 6, 2, 9, 7 \}$ ，建立对应的哈夫曼树



三、哈夫曼树的应用

哈夫曼树的应用很广，哈夫曼编码就是其在电讯通信中的应用之一。在电讯通信业务中，**通常用二进制编码来表示字母或其他字符**，并用这样的编码来表示字符序列。

三、哈夫曼树的应用

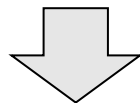
例：如果需传送的电文为 ‘ABACCDA’，它只用到四种字符，用两位二进制编码便可分辨。

假设A, B, C, D的编码分别为00, 01, 10, 11，则上述电文便为 ‘00010010101100’（共14位），译码员按两位进行分组译码，便可恢复原来的电文。

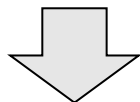
能否使编码总长度更短呢？

三、哈夫曼树的应用

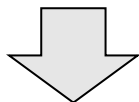
实际应用中各字符的出现频度不相同



用短（长）编码表示频率大（小）的字符



使得编码序列的总长度最小，使所需总空间量最少



数据的最小冗余编码问题

三、哈夫曼树的应用

若假设A, B, C, D的编码分别为0, 00, 1, 01, 则电文‘ABACCDA’便为‘000011010’（共9位）。

可译为‘BBCCDA’ ‘ABACCDA’ ‘AAAACCACA’
存在多义性

三、哈夫曼树的应用

译码的惟一性问题：要求任一字符的编码都不能是另一字符编码的前缀！这种编码称为**前缀编码**（其实是非前缀码）。

在编码过程要考虑两个问题 { 数据的最小冗余编码问题
译码的惟一性问题

利用最优二叉树可以很好地解决上述两个问题

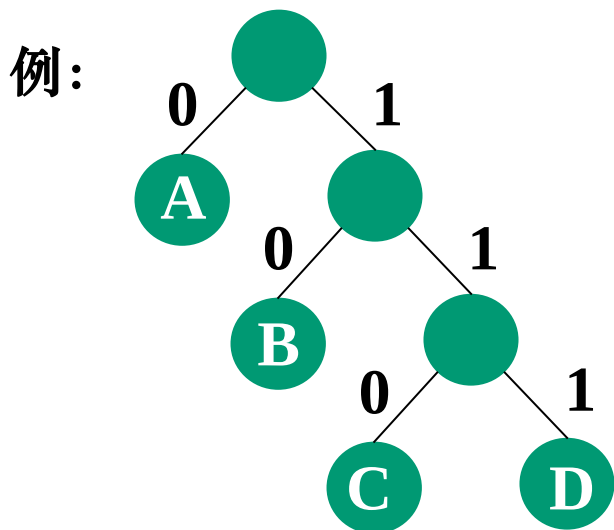
用二叉树设计二进制前缀编码

电文字符作为叶子结点构造二叉树

将二叉树中结点的左孩子分支标‘0’，右孩子分支标‘1’

每个字符的编码即为从根到每个叶子的路径上得到的0, 1序列

如此得到的即为二进制前缀编码



编码： A: 0
B: 10
C: 110
D: 111

任意一个叶子
结点都不可能
在其它叶子结
点的路径中。

用哈夫曼树设计总长最短的二进制前缀编码

假设各个字符在电文中出现的次数（或频率）为 w_i ，其编码长度为 l_i ，电文中只有 n 种字符，编码总长为：

$$WPL = \sum_{i=1}^n w_i l_i \quad \Rightarrow \quad \text{从根到叶子的路径长度}$$



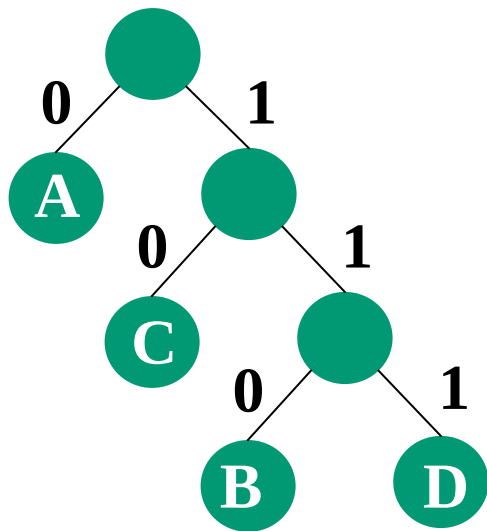
叶子结点的权

设计电文总长最短的编码 $\longleftrightarrow$ 设计哈夫曼树（以 n 种字符出现的频率作权）

由哈夫曼树得到的二进制前缀编码称为哈夫曼编码

例：如果需传送的电文为 ‘ABACCDA’，即：A, B, C, D 的频率（即权值）分别为0.43, 0.14, 0.29, 0.14，试构造哈夫曼编码。

解：

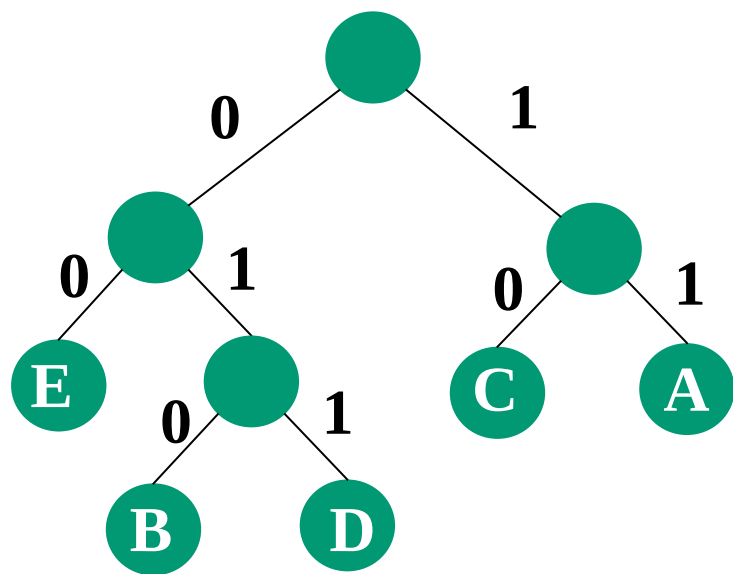


编码： A: 0
C: 10
B: 110
D: 111

则电文 ‘ABACCDA’ 便为 ‘0110010101110’（共13位）。

例：如果需传送的电文为 ‘ABCACCDAAEAE’，即：A, B, C, D, E的频率（即权值）分别为0.36, 0.1, 0.27, 0.1, 0.18，试构造哈夫曼编码。

解：



编码： A: 11
C: 10
E: 00
B: 010
D: 011

则电文 ‘ABCACCDAAEAE’便为 ‘110101011101001111001100’
(共24位，比33位短)。

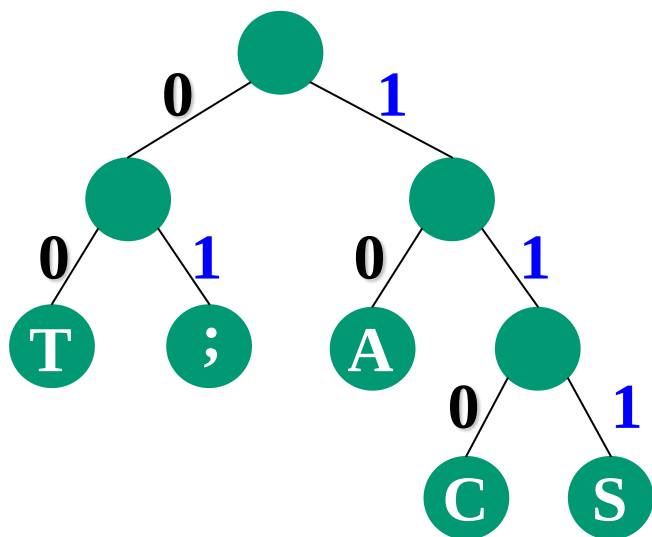
译码

从哈夫曼树根开始，对待译码电文逐位取码。

若编码是“0”，则向左走；

若编码是“1”，则向右走，

一旦到达叶子结点，则译出一个字符；再重新从根出发，直到电文结束。



电文为 “1101000”

译文只能是 “CAT”

建立哈夫曼树及求哈夫曼编码的算法

```
typedef struct {  
    unsigned int weight;  
    unsigned int parent,lchild,rchild;  
} HTNode, *HuffmanTree; //动态分配数组存储赫夫曼树  
typedef char **HuffmanCode;
```

```
void HuffmanCoding(HuffmanTree &HT,HuffmanCode &HC,int *w, int n)
{ //w存放n个字符的权值, 构造赫夫曼树HT,并求出n个字符的赫夫曼编码HC
  if (n<=1) return;    // n为字符数目

  m=2*n-1;             // m为结点数目

  HT = (HuffmanTree)malloc((m+1)*sizeof(HTNode));

  //HT存放Huffman树结构, 0号单元未用,其余前n个单元

  //存放树的叶子结点, n-1个单元存放内部结点
  for (p=HT[1], i=1; i<=n; ++i,++p,++w)
  { p->weight = *w; p->parent=0;
    p->lchild=0;  p->rchild=0; }
  // *p={*w,0,0,0}; 初始化HT中的叶结点循环退出时i=n+1;
```

```
for (; i<=m;++i,++p)
{ p->weight = 0; p->parent=0;
  p->lchild=0; p->rchild=0; }
// *p={0,0,0,0}; 初始化HT中的内部结点
for (i=n+1; i<=m;++i)
    // 建赫夫曼树, (建立HT静态链表中的链)
    { Select(HT,i-1,s1,s2); //在HT[1~i-1]中选择parent
      // 为0且weight最小的两个结点, 序号分别为s1和s2
      HT[s1].parent=i; HT[s2].parent = i;
      HT[i].lchild = s1; HT[i].rchild = s2;
      HT[i].weight = HT[s1].weight + HT[s2].weight;
    }
```

小结：哈夫曼树及其应用

1. 哈夫曼算法的思路：

——权值大的结点用短路径，权值小的结点用长路径。

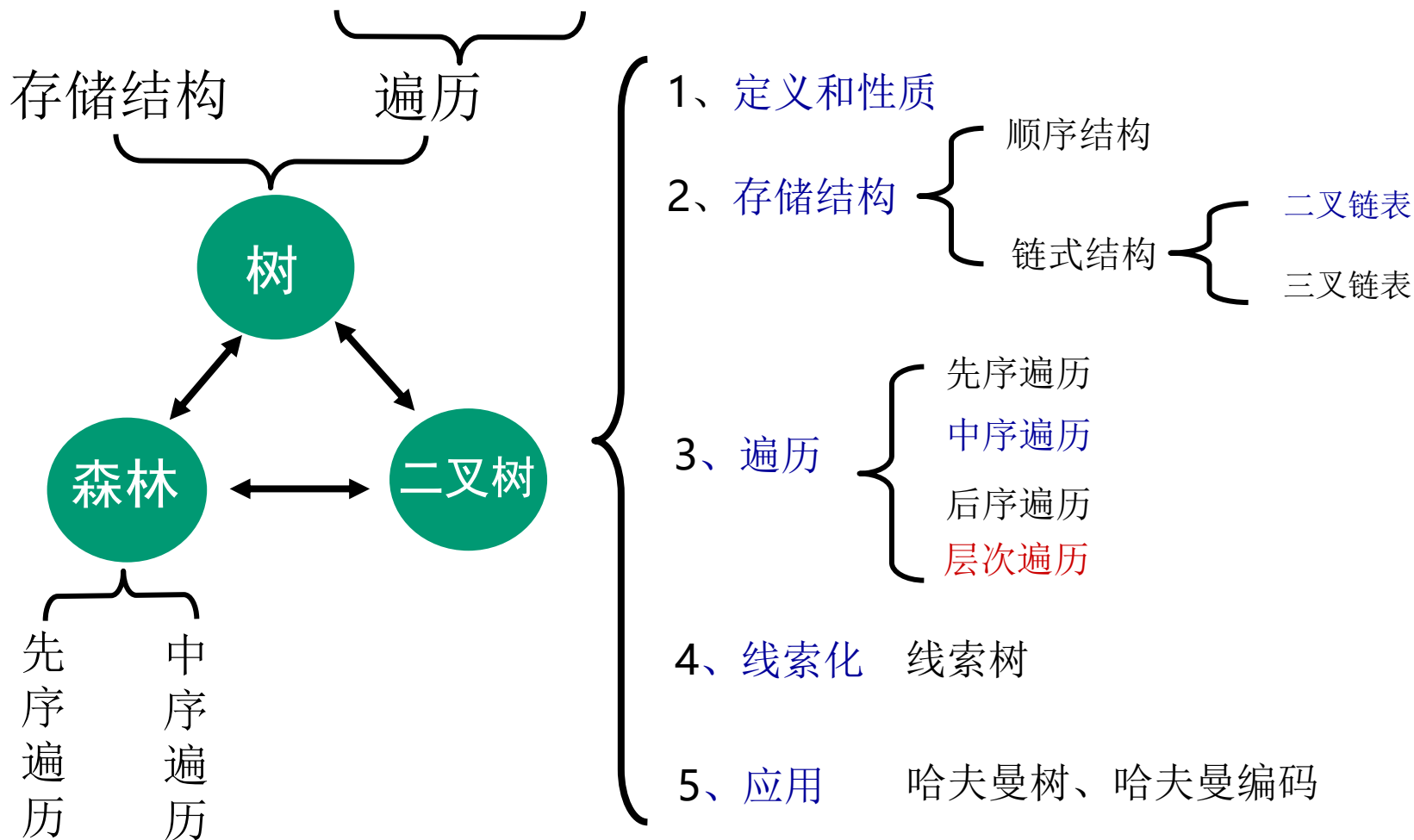
2. 构造哈夫曼树的步骤：

——对权值的合并、删除与替换

3. 哈夫曼编码规则： 左“0”右“1”

——又称为前缀码、最小冗余编码、紧致码等等，它是数据压缩学的基础。

孩子—兄弟 先根遍历 后根遍历



- 6.7
- 6.26
- 6.27
- 6.44,
- 6.65